EnergiQ AI: Cloud-Native Intelligent Energy Forecasting, Anomaly Detection and Optimisation Platform

Major project report

Uzair

03 November 2026

Table of Contents

Abstract....5

Chapter 1. Introduction and problem statement....6

Chapter 2. Objectives and outcomes....6

Chapter 3. Literature review....7

1. Load forecasting....8

2. Models for tabular time-series features....8

3. Evaluating time-series models....8

4. Anomaly detection....8

5. Explainability....9

6. Data....9

7. Gap addressed by this project....9

Chapter 4. Data: inventory, quality and suitability....9

1. Verdict....9

2. Provenance....9

3. File inventory....10

4. Columns....10

5. Time axis....10

6. Data quality....11

7. Requirement-by-requirement suitability....12

8. Limitations that must appear in the final report....12

9. Preprocessing policy (proposed in Phase 1; applied in Phase 3, see docs/05_ml_methodology.md)....12

10. Reproduce....13

Chapter 5. Exploratory data analysis....13
1. Headline findings....13
2. Clock check (the Phase 1 open question)....14
3. Data availability by proposed split....14
4. Decisions and proposals....15
5. Limitations....15
6. Reproduce....15
7. Chart palette....15
Chapter 6. Requirements....16
1. Functional requirements....16
2. Non-functional requirements....16
3. Explicit non-claims....17
4. Environment....17
Chapter 7. System architecture....17
1. System overview....18
2. Technology choices....18
3. Repository layout....18
4. ML data flow....19
5. Backend modules and endpoints (as built: docs/api/API_REFERENCE.md)....19
6. Database entities (as built: 12 tables, docs/diagrams/er_diagram.png)....19
7. Security approach (as built: docs/10_security.md)....19
8. Deployment status ladder....20
9. Key decisions and open questions....20
Chapter 8. Machine-learning methodology....20
1. Problem statement for the models....20
2. Forecasting framework....20
3. Models....21
4. Evaluation protocol....22
5. Reproducing the benchmark....22
Chapter 9. Forecast evaluation....23

1. Headline table (test MAE, kW; lower is better) 23
2. What the statistics support 23
3. Seed robustness of the LSTM 24
4. Honest reading of the error level 24
5. Application models versus the benchmark 24
6. Figures 25
7. Threats to validity 25
8. Reproducibility and library versions 25
Chapter 10. Explainability, cost and recommendations 26
1. Explainability 26
2. Tariff-based cost 27
3. Recommendations 27
4. Limitations specific to this module 28
Chapter 11. Anomaly detection 28
1. The central limitation: there are no labels 28
2. Method 28
3. Results on the real dataset (medium sensitivity) 29
4. Behaviour you should expect 30
5. Synthetic injection check (clearly synthetic) 30
6. In the application 30
Chapter 12. Backend design and API 30
1. Layers 30
2. Data model (docs/diagrams/er_diagram.png) 31
3. Background jobs 31
4. Upload pipeline 31
5. Forecast and explanation endpoints 32
6. Observability endpoints 32
Chapter 13. Web interface 32
1. Stack and structure 33
2. Design decisions that matter 33

3. Verification (what was run, with results) 34
4. Known gaps and risks 34
Chapter 14. Security 35
1. Controls and where they are verified 35
2. Static and dependency scans (run in this project) 37
3. What was not done 37
4. Privacy 37
Chapter 15. Testing 37
1. Test suites 38
2. Properties the tests pin down (not just “it runs”) 38
3. Security tests (tests/integration/test_security.py, 16 tests) 39
4. Measured performance and resources 39
5. Browser end-to-end test 40
6. Defects found during verification 40
7. Not tested 40
Chapter 16. Deployment and operations 41
1. Status ladder (what has really been done) 41
2. Local development (Windows, macOS, Linux) 41
3. Local production-like stack (Docker Compose) 42
4. Database migrations 42
5. AWS (configuration only) 42
6. Configuration reference 43
Chapter 17. MLOps and monitoring 43
1. What is tracked 43
2. Model monitoring (GET /api/v1/models/{id}/monitoring) 43
3. Service monitoring 44
4. Backup and restore 44
5. Release and change process 44
Chapter 18. Limitations and future scope 45
1. Limitations (what a reader should not over-trust) 45

2. Future scope....46
Chapter 19. Verification matrix....46
1. Deployment level (the ladder from docs/03_architecture.md §8)....46
2. Functional requirements....47
3. Non-functional requirements....48
4. Phases of the specification....49
5. Academic deliverables (spec §12)....49
6. Things only you can do....50
Appendix A. Design diagrams....51
Appendix B. Screenshots of the running application....56
References....65

Abstract

Households and small buildings increasingly record electricity use at fine time resolution, but turning such records into reliable forecasts, understandable alerts and actionable cost advice normally needs separate tools and specialist skills. This project builds EnergiQ AI, an end-to-end web platform that takes a user's meter data and offers (i) validated ingestion with a data-quality score, (ii) 24-hour-ahead load forecasts with an uncertainty range, (iii) transparent detection of unusual consumption, (iv) exact feature-level explanations of forecasts, and (v) tariff-based cost analysis with rule-based recommendations and a cheapest-start scheduler.

The forecasting models were developed and evaluated on the public UCI household electric power consumption data set (one French household, 34,151 valid hourly values, December 2006 to November 2010). Hourly load was predicted 1, 6 and 24 hours ahead with direct multi-horizon models, using only information available at the forecast origin, chronological splits, expanding-window cross-validation for model selection, and a final test period (January to November 2010) scored once. XGBoost and Random Forest reduced the mean absolute error by about 18 % (1 h) and 23 to 25 % (6 h and 24 h) relative to the strongest baseline chosen by cross-validation (for example 0.336 kW for XGBoost and 0.333 kW for Random Forest against 0.408 kW at 1 h); all differences from the baseline were significant under a paired day-block bootstrap. An LSTM neural network was no better than the tree models (worse at 1 h, indistinguishable at 6 and 24 h), and the explained variance at longer horizons stayed modest (R^2 about 0.40), which the report states plainly. Because the data set has no fault labels, anomaly detection is evaluated only through a clearly labelled synthetic injection test and no accuracy figure is claimed.

The system is implemented as a FastAPI service with role-based access, background jobs, database migrations (SQLite for development, PostgreSQL for production), a React and TypeScript interface, Docker and GitHub Actions configuration, and AWS CloudFormation templates. The software was tested with automated unit, integration, security and front-end tests, including runs on PostgreSQL 16. Container images and the AWS deployment were configured but not executed in the development environment, and the report documents this boundary together with the limitations of a single-household evaluation.

Keywords: load forecasting, XGBoost, anomaly detection, explainable AI, TreeSHAP, smart meter data, FastAPI, MLOps.

Chapter 1. Introduction and problem statement

Context. Smart meters and loggers produce electricity-use records for homes and small sites. The raw numbers are rarely used: people see a monthly bill but not what drives it, when an unusual period occurred, or when it would be cheaper to run flexible loads.

Problems addressed.

1. *Data trust.* Meter exports vary in format, unit, delimiter and time convention, and contain gaps and stuck readings. Forecasts built on unvalidated data are unreliable and users cannot tell why.
2. *Forecasting without leakage.* Many demonstrations evaluate time-series models with random splits or features that quietly contain future information, which overstates accuracy. A credible tool must evaluate chronologically and report honest errors against simple baselines.
3. *Unlabelled anomalies.* Real consumption data has no ground-truth fault labels, so "anomaly detection" is easy to oversell. A credible tool must say what it flags, why, how often, and what it cannot claim.
4. *Opaque models.* A forecast without an explanation is hard to trust or act on.
5. *From insight to action.* Forecasts and alerts matter only if they connect to cost and to decisions the user can make, with assumptions made explicit.
6. *Engineering gap.* Notebooks that show a model are not a system a person can use: they lack authentication, validated upload, job handling, persistence, monitoring, migrations, tests and a deployment path.

Problem statement. Design, implement and evaluate a secure, tested, cloud-ready web platform that ingests household electricity data, forecasts its hourly load 1 to 24 hours ahead with honestly measured accuracy, detects and explains unusual consumption without claiming unmeasurable accuracy, explains individual forecasts, and converts the results into tariff-based cost analysis and clearly labelled scenario recommendations, while recording exactly which parts have and have not been deployed or verified.

Scope. One-meter, hourly-resolution household data; CPU-only training on an 8 GB machine; a public data set for benchmarking; user-supplied tariffs.

Out of scope. Real-time streaming from meters, appliance-level disaggregation, weather and occupancy sensing, grid-scale forecasting, billing integration, a mobile app, and any claim of performance on other households.

Chapter 2. Objectives and outcomes

#	Objective	Success criterion (stated before evaluation)	Delivered in	Outcome
O1	Validated ingestion and quality scoring	Wrong header, unparseable time, non-numeric value, binary file and oversize file are rejected with a reason; gaps and flat runs reported; nothing	`ml_engine/data/`, `backend/app/services/data.py`; tests `test_loading`, `test_api_core`	Met (automated tests)

#	Objective	Success criterion (stated before evaluation)	Delivered in	Outcome
		imputed silently		
O2	Leak-free forecasting benchmark	Chronological splits only; features use data at or before the origin; test period scored once; common evaluation rows; baselines included	`docs/05_ml_methodology.md`; `test_features_splits`, `test_forecasting`, `test_forecast_uses_only_data_up_to_the_origin`	Met
O3	Models that beat baselines, with uncertainty	Learned models beat the cross-validation-chosen baseline at 1, 6 and 24 h, with a bootstrap interval excluding zero	`docs/06_model_evaluation.md`	Met for XGBoost, Random Forest and LSTM at all three horizons. Accuracy is modest ($R^2 \approx 0.40$ at 6 and 24 h)
O4	Compare a lightweight deep model	CPU PyTorch LSTM trained under the same protocol, three seeds, headline seed fixed in advance	`ml_engine/forecasting/lstm.py`, `scripts/run_lstm.py`	Done. Result: no demonstrated benefit over trees
O5	Anomaly detection with honest evaluation	Explained events, stated alert rate, synthetic check labelled as synthetic, no accuracy claim	`docs/08_anomaly_detection.md`	Delivered; accuracy on real faults is **not measurable** with this data
O6	Explainable forecasts	Exact TreeSHAP with verified additivity; plain-language groups	`docs/07_…`; `test_explain_optimize`	Met
O7	Cost analysis and recommendations	Tariff-based cost, scenario savings with assumptions, cheapest-window scheduler; demo tariff labelled	`ml_engine/optimization/`	Met (scenario estimates, not measured savings)
O8	Secure multi-user backend	Authentication, roles, ownership checks, validated uploads, rate limits, audit	`docs/09_…`, `docs/10_security.md`; 16 security tests	Met, no external penetration test
O9	Usable web interface using only real API data	All main workflows available; no hard-coded numbers	`frontend/`	See `docs/15_verification_matrix.md`
O10	Reproducible deployment path	Migrations, Docker files, CI, AWS templates validated statically	`docs/12_deployment.md`	Configured and statically checked; **not built or deployed here**
O11	Monitoring and operations	Health, metrics, drift and realised-error checks, backups tested	`docs/14_mlops_monitoring.md`; `test_backup_restore`	Met locally
O12	Honest reporting	Limitations and unverified items stated; deployment status ladder	`docs/13_limitations.md`, `docs/15_verification_matrix.md`	Met

Chapter 3. Literature review

Numbers in square brackets refer to `docs/references.md`, where each source was checked against a public record. The review is deliberately short and covers only what the project relies

on. The project is an engineering integration of established methods; it does **not** claim a new algorithm.

1. Load forecasting

Electric load forecasting is a long-established field. Hong and Fan [13] give a tutorial review of probabilistic load forecasting and stress that point forecasts alone hide uncertainty and that evaluation must match the use of the forecast. Hyndman and Athanasopoulos [8] provide the standard practical reference for forecasting methods and for benchmarking against simple methods such as the naive and seasonal-naive forecast, which this project adopts as baselines. Hyndman and Koehler [7] review accuracy measures and warn that percentage errors behave badly near zero; this is why the project reports mean absolute error in kW as the headline and lists MAPE only for completeness. (Their scaled measure, MASE, is a reasonable alternative that this project does not report.)

Individual-household load is harder to forecast than aggregated load because it is dominated by a few occupants' routines and appliances. Kong et al. [12] proposed an LSTM recurrent network [3] for short-term forecasting of individual households and reported good results on their data, which motivated including an LSTM here.

2. Models for tabular time-series features

Random forests [1] and gradient-boosted trees as implemented in XGBoost [2] are standard supervised learners for tabular data. Grinsztajn et al. [10] report that tree-based models remain very competitive with, and often better than, deep networks on typical medium-sized tabular data sets. This is consistent with the project's finding that the LSTM did not outperform the tree models. The project frames forecasting as supervised learning on lagged, rolling and calendar features and trains one model per horizon (direct strategy), implemented with scikit-learn [15] and XGBoost [2]; the LSTM was written with PyTorch [16].

3. Evaluating time-series models

Random shuffling lets a model train on the future of the period it is tested on. Bergmeir and Benítez [9] study how cross-validation can be applied to time-series predictors and compare it with out-of-sample evaluation, and Cerqueira et al. [11] compare performance-estimation methods empirically across many series, reporting that the best choice depends on the properties of the series (notably stationarity) and that methods which respect temporal order are the safer choice when the series changes over time. The project therefore uses expanding-window cross-validation for model selection and a final, chronologically later test period scored once, and it reports uncertainty with a paired day-block bootstrap so that autocorrelation within a day is respected.

4. Anomaly detection

Chandola et al. [14] survey anomaly detection and distinguish point, contextual and collective anomalies. A household reading is typically *contextual*: 2 kW may be normal at 19:00 and unusual at 03:00. The project's detector therefore measures deviation from a causal, hour-of-week baseline and compares it with the empirical residual distribution for that hour of day. Isolation Forest [4], which isolates unusual points by random partitioning and needs no labels, is used as corroborating evidence. The survey also underlines a difficulty this project meets directly: without labelled anomalies, detection quality cannot be measured on real data, so only the alert rate and a synthetic injection test are reported.

5. Explainability

SHAP [5] attributes a prediction to features using Shapley values from cooperative game theory, and TreeSHAP [6] computes these exactly and efficiently for tree ensembles. This is why the explained model in the application is XGBoost and why the project can verify additivity numerically. SHAP explains the model, not the household: attribution is not causation.

6. Data

The UCI "Individual household electric power consumption" data [17] contains about four years of one-minute readings from one house near Paris, with missing values marked in the file. It is widely used, public, and licensed CC BY 4.0, and it is the only data used for the benchmark. Its limitations (one household, one location) are discussed in `docs/13_limitations.md`.

7. Gap addressed by this project

The cited work provides methods and evaluation principles, but a user wanting to apply them to their own meter data must assemble ingestion, quality control, leak-free evaluation, explanation, cost translation, security, operations and deployment themselves. EnergiQ AI is an attempt to integrate those pieces transparently and to be explicit about what each piece can and cannot claim. Its contribution is engineering integration and honest evaluation on one public data set, not a new forecasting method.

Chapter 4. Data: inventory, quality and suitability

Phase 1 deliverable. Every number below was measured from the supplied file by `scripts/profile_dataset.py` (machine-readable output: `artifacts/reports/dataset_profile.json`). Units, sub-meter coverage and collection site come from the official UCI dataset page, because the file itself carries no unit metadata. Nothing here is estimated or invented.

1. Verdict

SUITABLE for electricity-consumption forecasting, anomaly screening and tariff-based cost analysis on a single household. It is **not** suitable for weather-driven modelling, supervised anomaly evaluation, or any claim about generalising across households (see section 8).

2. Provenance

Item	Value
Name	Individual Household Electric Power Consumption
Creators	Georges Hebrail, Alice Berard
Donated to UCI	29 August 2012
Licence	Creative Commons Attribution 4.0 International (CC BY 4.0) — attribution required
DOI	10.24432/C58K54
Collection site	One house in Sceaux, about 7 km from Paris, France (per UCI)
Period (per UCI)	December 2006 – November 2010
Citation	Hebrail, G. & Berard, A. (2006). *Individual Household Electric Power Consumption* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C58K54
Access	Public download, no registration required
Supplied file	`household_power_consumption.txt` (from your zip)
SHA-256	`4259c9d7ece5dbee9ab8d53682baac68d791c864f0f`

Item	Value
	`64a52b4043cb3b90894b7`

Independent check: UCI publishes 2,075,259 instances; `wc -l` on the supplied file gives 2,075,260 lines including the header, i.e. 2,075,259 data rows.

3. File inventory

Property	Measured value
Format	Semicolon-delimited text, one header line
Size	132,960,755 bytes (about 127 MB)
Data rows	2,075,259
Columns	9
Fields per line	exactly 9 on all 2,075,260 lines (no malformed lines)
Header matches expected schema	Yes
Unparseable timestamps	0
Profiling cost	about 15 s, 464–508 MB peak RAM (two runs), chunked read

4. Columns

Date format is `dd/mm/yyyy`; Time is `hh:mm:ss`. Statistics are over the 2,049,280 rows that contain readings.

Column	Unit (per UCI)	Min	Mean	Max	Notes
Global_active_power	kW	0.076	1.092	11.122	Primary forecasting target. No zeros, no negatives.
Global_reactive_power	kW (as documented)	0.000	0.124	1.390	481,561 exact zeros (23.5%)
Voltage	V	223.20	240.84	254.15	Plausible mains range
Global_intensity	A	0.2	4.63	48.4	
Sub_metering_1	Wh	0	1.12	88	Kitchen (dishwasher, oven, microwave). 91.7% zeros.
Sub_metering_2	Wh	0	1.30	80	Laundry (washer, dryer, fridge, light). 70.1% zeros.
Sub_metering_3	Wh	0	6.46	31	Electric water heater and air conditioner. 41.6% zeros.

All three sub-meter columns contain only whole numbers (1 Wh resolution per minute), so they are coarse for low loads.

5. Time axis

Property	Measured value
First reading	2006-12-16 17:24:00
Last reading	2010-11-26 21:02:00
Sampling interval	60 s; all 2,075,258 consecutive steps are exactly 60 s
Chronologically sorted	Yes
Duplicate timestamps	0
Absent timestamps (rows missing from the span)	0 — span coverage 100%
Complete calendar years	2007, 2008, 2009 only (2006 starts 16 Dec; 2010 ends 26 Nov)

Clock behaviour (resolved in Phase 2, see `04_eda_report.md` section 2): every step is exactly 60 s, each of the 8 EU clock-change days has exactly 1,440 rows, and the household's weekday load profile does not shift by an hour at any of them. So the timeline has no skipped or repeated hour and the routine stays at the same logger-clock time all year. How the provider produced this is not documented.

6. Data quality

6.1 Missing values

Property	Measured value
Rows with no readings	25,979 (1.2518% of rows)
Pattern	All seven measurement columns are missing together on the same rows; there are no partially missing rows
Contiguous missing blocks	71
Block length	median 1 min, 95th percentile 3,217 min
Blocks longer than 1 hour	9
Blocks longer than 1 day	6

Longest blocks:

Start	Length
2010-08-17 21:02	7,226 min (about 5.0 days)
2010-09-25 03:56	5,237 min
2007-04-28 00:21	3,723 min
2009-06-13 00:30	3,305 min
2010-01-12 14:53	3,129 min
2010-03-20 03:52	2,027 min

Missing rows as a share of rows present, by year:

2006	2007	2008	2009	2010
0.018%	0.748%	0.026%	0.814%	3.711%

Four of the six longest blocks fall in 2010. The most natural chronological test period is therefore the worst-quality period; test scoring must report how many hours were excluded (section 9).

Documentation vs. file. The UCI page says missing values appear as empty fields. In this file the missing marker is `?` in the first six numeric columns and an empty field in `Sub_metering_3` (the trailing field). The ingestion parser must accept both tokens.

6.2 Invalid and unusual values

Check	Result
Non-numeric tokens other than the missing markers	0
Negative readings	0 in every column
Global_active_power at or below zero	0
Global_active_power above the 1.5×IQR fence (3.358 kW)	95,070 readings (4.6%)
Global_active_power above the 3×IQR fence (5.188 kW)	14,413 readings (0.70%)
Energy bookkeeping: minutes where sub-meters exceed total active energy	1,050 of 2,049,280 (0.051%)

The IQR counts are a screen, not a finding: high household load is expected when several appliances run together, so these are not treated as errors. The 1,050 bookkeeping minutes are small metering or rounding inconsistencies.

6.3 Energy coverage of the sub-meters

Using UCI's formula (`Global_active_power × 1000 / 60 − Sub_metering_1 − _2 − _3`, in Wh per minute), the mean unmetered energy is 9.31 Wh/min against a mean total of 18.19 Wh/min: about **51% of measured active energy is not attributed to any sub-meter**. Appliance-group explanations must not be presented as a full breakdown of household use.

6.4 Usable data after aggregation to hourly means

Property	Measured value
Hours in span	34,589
Hours with all 60 minutes valid	34,085 (98.5%)
Hours with at least 45 valid minutes	34,151 (98.7%)
Hours with zero valid minutes	421
Days in span	1,442
Days with at least 95% valid minutes	1,416 (98.2%)
Days with no valid minutes	9

7. Requirement-by-requirement suitability

Spec requirement	Supported by this data?	Basis
Timestamps	Yes	100% parseable, regular 60 s grid
Consumption target	Yes	`Global_active_power` (kW); hourly energy in kWh = mean kW × 1 h
Horizons: next hour, 6 h, 24 h	Yes	34,151 hourly points with at least 45 valid minutes
Seasonal naive baselines (24 h, 168 h)	Yes	Three complete calendar years
Hour, day, weekday, month, weekend, lags, rolling statistics	Yes	Derived from the timestamp and target
Weather features	**No**	No weather column; none will be created
Tariff-based cost and savings	**No data**	Dataset has no tariff; tariffs will be user-entered. Any default tariff is labelled demo and is not a savings claim
Verified anomaly labels	**No**	No labels; evaluation is unsupervised only
Calendar features such as public holidays	Possible	Needs a documented French holiday source; **not implemented** (listed as future work)
Multiple households or users	**No**	One household

8. Limitations that must appear in the final report

1. One household: results say nothing about other homes, and cross-household generalisation cannot be claimed.
2. Anomalies found without labels are unconfirmed unusual readings, not faults, theft or equipment failure.
3. About half of active energy is unmetered, so sub-meter explanations are partial.
4. 2010 has the most missing data (3.7%), including a block of about 5 days whose cause is unknown.
5. Data ends in November 2010, so patterns reflect that period only.

9. Preprocessing policy (proposed in Phase 1; applied in Phase 3, see `docs/05_ml_methodology.md`)

- Canonical series: hourly mean of `Global_active_power`, built only from hours with at least 45 valid minutes; other hours stay missing.
- Missing values are never filled in the evaluation target. Short gaps may be interpolated for model inputs only, never for scored values.

- Chronological split (proposal): train 2006-12-16 to 2009-06-30, validation 2009-07-01 to 2009-12-31, test 2010-01-01 to 2010-11-26. The test set lacks December, and the report must state the number of excluded test hours.
- All scalers and encoders are fitted on the training split only.

10. Reproduce

```
python scripts\profile_dataset.py --input data\raw\uci_household_power\
household_power_consumption.txt --output artifacts\reports\
dataset_profile.json
```

Phase 2 adds exploratory plots (daily and weekly profiles, seasonality, missing-block timeline, the daylight-saving check).

Chapter 5. Exploratory data analysis

> **Status update (end of project).** This report was written in Phase 2 and is kept as written; the proposals in §4 were resolved as follows. Validation: the expanding-window cross-validation was adopted (8 quarterly folds). Features: the final set differs from the §4 proposal (lags 0, 1, 2, 3, 5, 11, 23, 47 plus values at the same time yesterday and last week, rolling statistics, calendar features; see `docs/05_ml_methodology.md`). The 401-minute flat run led to the identical-readings rule in the anomaly detector (`docs/08_anomaly_detection.md`). **Not done:** the with/without sensitivity check for the August 2008 low-use block was never run, so the effect of that atypical period on the models is unquantified.

All numbers come from `scripts/run_eda.py` run on the real UCI file (`artifacts/reports/eda/eda_stats.json`; figures in `artifacts/reports/eda/`). The hourly series follows the Phase 1 policy: hourly mean of `Global_active_power`, kept only when the hour has at least 45 valid minutes (34,151 hours; 438 hours are left empty, versus 504 under a strict 60-minute rule). "Logger clock" means the timestamps exactly as stored in the file.

1. Headline findings

1. **Level and stability.** Mean hourly load is 1.092 kW (median 0.803, 95th percentile 2.866, highest hour 6.561 kW on 2008-11-23 18:00). That is about 25–27 kWh/day. Over the same calendar window (1 Jan – 25 Nov) the yearly means are 25.58 (2007), 25.13 (2008), 25.22 (2009) and 25.44 (2010) kWh/day, a spread of under 2%, so there is no visible multi-year trend.
2. **Strong daily routine, different on weekends.** Figure 2 and Figure 3. Weekdays have a sharp morning rise to about 1.7 kW at 07:00 and a larger evening peak of 1.86 kW at 21:00, with a 03:00 trough of 0.41 kW. Weekends have no sharp morning peak, a broad midday plateau near 1.4–1.5 kW and an evening peak of 2.05 kW at 19:00. Weekend mean load is 1.234 kW against 1.036 kW on weekdays (about 19% higher).
3. **Strong yearly seasonality.** Figure 4. Mean daily energy is about 33 kWh/day in December–February and about 18 kWh/day in June–August (Aug 2008 excluded), roughly 1.8 times higher in winter. The data contains no temperature, so the cause (heating, lighting, occupancy) cannot be established from this dataset.
4. **Unusual low-use period.** The ten lowest-energy days are all 20–29 August 2008, at 4.17–4.47 kWh/day, giving an August 2008 monthly mean of 6.6 kWh/day (other Augusts: 14–18). This is consistent with the house being largely unoccupied, but that is an interpretation, not something the data states.

5. **Memory of the series.** Figure 5. Hourly autocorrelation is 0.71 at lag 1, 0.44 at lag 24 and 0.46 at lag 168 (0.37–0.41 at the other daily multiples up to 6 days). The daily rhythm is real but day-to-day variation is large, so long-horizon skill is expected to be moderate. This is a hypothesis for Phases 4–7, not a result. Long-lag values also include the yearly seasonal level, which inflates them slightly.
6. **Skewed, two-mode distribution.** Figure 6. One mode near 0.3 kW (night and idle hours) and a second near 1.4–1.5 kW, with a long right tail. Error metrics must therefore be reported alongside scale-free ones (sMAPE) and per-regime, not just RMSE.
7. **About half the energy is not sub-metered.** Figure 7. 51.2% of measured active energy is outside the three sub-meters. Of the sub-metered part, sub-meter 3 (water heater and air conditioner) is clearly the largest and sub-meter 1 (kitchen) the smallest.
8. **Top demand hours are winter evenings.** All ten highest hours fall between November and February, between 17:00 and 21:00.
9. **One suspicious flat run.** Exactly one stretch of 30 minutes or longer has identical readings: 401 minutes at 0.360 kW starting 2008-10-06 10:09. Every other run of identical readings is 29 minutes or shorter. This is a candidate data artifact (for example a stuck sensor), unconfirmed.

2. Clock check (the Phase 1 open question)

I tested whether the logger clock follows daylight saving, in two independent ways.

- **Row counts.** On all 8 EU clock-change days (last Sunday of March and October, 2007–2010) the file has exactly 1,440 rows, and every row in the 02:00 hour has a real reading. A clock that follows daylight saving would be missing the 02:00 hour in spring or repeating it in autumn.
- **Behaviour.** For each change I compared the weekday load profile in the two weeks before and after (10-minute resolution, best of ±3 h shifts). Figure 8. The estimated shift is 0 or +10 minutes at all 8 changes, against −20 to +10 minutes for 16 control comparisons between adjacent windows with no clock change. The detector recovers injected shifts of ±30 and ±60 minutes exactly, so an hour-long step would have been seen.

Conclusion. The household's routine sits at the same logger-clock time before and after every clock change, and the timeline has no skipped or repeated hour. Hour-of-day features are therefore consistent across the whole year. How the data provider produced this continuous, routine-aligned clock is not documented and remains unexplained. It does not block modelling, but ingestion of user uploads must not assume the same: users will declare their timestamp convention (Phase 3 and Phase 10).

3. Data availability by proposed split

Split	Period	Hours	Valid hours	Excluded	Mean kW	95th pct kW	Max kW	Mean kWh/day
Train	2006-12-16 to 2009-06-30	22,255	22,127 (99.42%)	128	1.119	3.002	6.561	26.85
Validation	2009-07-01 to 2009-12-31	4,416	4,401 (99.66%)	15	1.009	2.657	5.115	24.20
Test	2010-01-01 to 2010-11-26	7,918	7,623 (96.27%)	295	1.061	2.574	5.627	25.44

Two consequences:

- The validation block (July–December) contains no winter peak months, so hyperparameters chosen on it alone would not be tuned for January–March demand.
- The test period has fewer extreme hours than training (95th percentile 2.57 vs 3.00 kW) and loses 3.7% of its hours to missing data. Part of this is seasonal mix, part may be a real difference between years; the data cannot separate them.

4. Decisions and proposals

Item	Status	Detail	Reason
Target	Confirmed (approved in Phase 1)	Hourly mean `Global_active_power`	Regular, strong daily and weekly structure
Hour validity	Confirmed (approved in Phase 1)	At least 45 valid minutes	Drops 438 hours; the strict rule would drop 504
Test period	Confirmed (approved in Phase 1)	2010-01-01 to 2010-11-26, untouched until final evaluation	Chronologically last
Validation	**Proposed change**	Replace the single July–December block with expanding-window time-series cross-validation over train plus validation, folds spanning different seasons	One block is not seasonally representative
Aug 2008 low-use block	Proposed	Keep in data; run a with/without sensitivity check in Phase 7	Real household behaviour, but atypical
401-minute flat run	Proposed	Flag as suspect in Phase 3; Phase 8 detector gets an identical-readings rule	Possible sensor artifact, unconfirmed
Features	Proposed	Hour, weekday, weekend, month; lags 1, 2, 3, 24, 48, 168; rolling mean and standard deviation over 24 h and 168 h; no weather; no holiday feature unless a documented source is added	Supported by the autocorrelation and by what the data contains
Baselines	Per specification	Naive; seasonal naive at 24 h and 168 h	Autocorrelation at both is similar (0.44, 0.46)

5. Limitations

- One household. Nothing here generalises to other homes.
- EDA describes patterns; it does not prove causes (no weather, occupancy or appliance data beyond the three sub-meters).
- The clock behaviour is measured, not explained.
- The figures are static PNGs for the report. The application's charts (Phase 11) will be built from the real API.
- Not yet run on Windows.

6. Reproduce

```
python scripts\run_eda.py --input data\raw\uci_household_power\
household_power_consumption.txt --outdir artifacts\reports\eda
python -m pytest
```

7. Chart palette

Colours come from a validated categorical palette (blue, orange, aqua, yellow, in that fixed order). The validator passes all hard checks (lightness band, chroma, colour-blind separation, normal-vision separation). It warns that aqua and yellow are below 3:1 contrast on the light

background, so every chart that uses them also carries a legend and direct labels, and the underlying numbers are in `eda_stats.json`.

Chapter 6. Requirements

Derived from the project specification. Statuses were last updated at the end of the project and only say what was actually verified; the evidence for each is in `15_verification_matrix.md`.

Status key: PASS, PARTIAL, NOT TESTED, NOT TESTABLE (defined in `15_verification_matrix.md`).

1. Functional requirements

ID	Requirement	Phase	Status
FR-01	Register and log in	10, 11	PASS
FR-02	Upload electricity consumption datasets (CSV)	10, 11	PASS
FR-03	Validate and explore uploaded data (quality report)	3, 10	PASS
FR-04	View historical consumption patterns	11	PASS
FR-05	Forecast future consumption (1 h, 6 h, 24 h)	4–7	PASS
FR-06	Compare forecasting models	7	PARTIAL (benchmark compares all models; the app compares with baselines)
FR-07	Detect abnormal consumption	8	PASS (function); accuracy NOT TESTABLE, no labels
FR-08	Explain predictions (Explainable AI)	9	PASS
FR-09	Estimate cost with editable tariff rules	9, 10	PASS
FR-10	Transparent optimisation recommendations, separate from anomalies	9	PASS
FR-11	View alerts and historical analysis	10, 11	PASS
FR-12	Export reports	10, 11	PASS
FR-13	Manage datasets and forecasting experiments	10, 11	PASS
FR-14	Monitor model performance	10, 14	PASS
FR-15	Admin panel and settings	11	PARTIAL (admin yes; no general settings page)

Dataset-dependent items (FR-05 to FR-10) rest on the supplied dataset, which passed the suitability check (`01_dataset_suitability.md`).

2. Non-functional requirements

ID	Requirement	How it is checked	Status
NFR-01	Runs on 8 GB RAM, CPU only, Windows	Peak memory measured per pipeline stage and recorded in reports	PASS for memory and CPU (peaks 0.3–1.1 GB); Windows NOT TESTED
NFR-02	No full-dataset loads when avoidable; chunked CSV processing	Profiler and ingestion read in chunks; peak RSS reported	PASS
NFR-03	Small default footprint: SQLite locally, one process, no Redis or Celery unless justified	Default `docker-compose` profile starts only backend, frontend and database	PASS locally; compose NOT TESTED

ID	Requirement	How it is checked	Status
NFR-04	Reproducible training (fixed seeds, versioned config, data hash)	Re-running training reproduces metrics; test in Phase 7	PASS with pinned libraries (XGBoost varies across versions)
NFR-05	No data leakage; chronological splits only	Leakage tests in Phase 3	PASS
NFR-06	Password hashing, authentication, authorisation, per-user data isolation	Security tests in Phase 12	PASS
NFR-07	Upload limits, safe file handling, CSV validation	Upload security tests in Phase 12	PASS
NFR-08	No secrets in code, logs or Git	`.env` ignored, `.env.example` only, secret scan in CI	PASS locally; CI job not yet run
NFR-09	Demo data always labelled as demo; no hard-coded statistics in the real UI	Frontend review in Phase 11	PASS (lint rule, review, tests)
NFR-10	Honest status reporting for deployment levels	Ladder in `03_architecture.md` section 8	PASS

3. Explicit non-claims

The platform and its report will not:

- present unlabelled anomalies as confirmed faults, theft or equipment failure;
- promise guaranteed energy or cost savings;
- invent weather, tariff or label data that the dataset does not contain;
- claim performance on any data other than the real held-out test split;
- claim cloud deployment without evidence from a successful deployment;
- create paid cloud resources without your explicit approval.

4. Environment

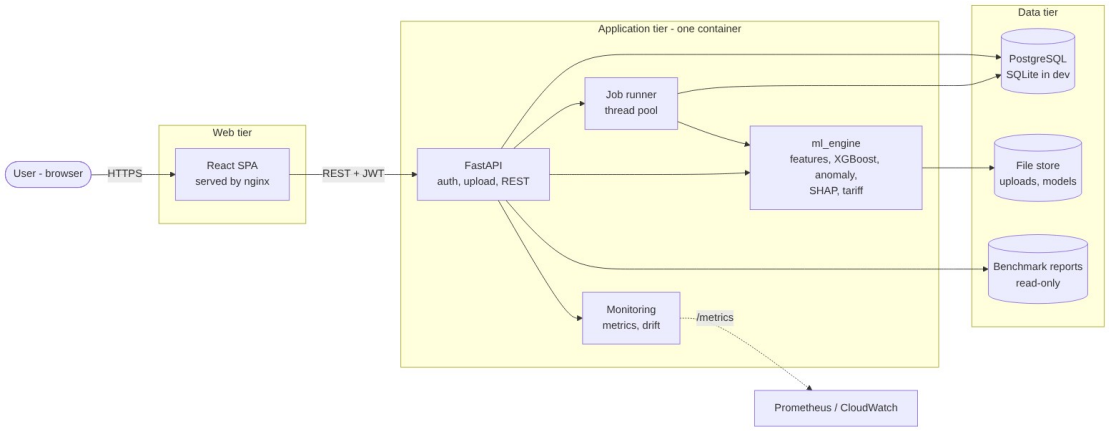
Item	Your machine	This sandbox
OS	Windows	Linux
Python	3.11 (Anaconda)	3.13 default; 3.11 also installed
RAM	8 GB	7 GB
GPU	none	none
Node	Node 20 or 22 LTS	22
Docker	needed to build the images	daemon starts, but images could not be built (registry blocked)

Anything verified only here is reported as “verified on Linux”, never as verified on Windows.

Chapter 7. System architecture

Originally written as the Phase 1 design and updated after implementation: sections marked *(as built)* describe what exists and was tested; where the build departed from the plan the text says so. Rendered diagrams are in `docs/diagrams/`; the detailed backend description is `docs/09_backend_and_api.md`.

1. System overview



System architecture

The ML code lives in its own package (`ml_engine`) with no web dependencies, so it can be tested, run from scripts and called by the backend without coupling.

2. Technology choices

Layer	Choice	Reason
Frontend	React, TypeScript, Vite, Tailwind CSS, Recharts	Per spec
Backend	Python 3.11, FastAPI, Pydantic, SQLAlchemy, Alembic	Per spec
Database	SQLite for local, PostgreSQL for production, same SQLAlchemy models	Per spec; keeps laptop light
Auth	argon2id password hashing, HS256 JWT access tokens, role-based authorisation (*as built*)	Standard, no extra service
Forecasting	Naive and seasonal naive, Random Forest, XGBoost, small PyTorch LSTM (CPU)	Per spec; baselines first
Anomaly detection	Causal hour-of-week baseline with empirical per-hour band, flat-run rule and Isolation Forest corroboration (*as built*, `docs/08_anomaly_detection.md`)	Per spec; no labels exist
Explainability	Exact TreeSHAP (XGBoost `pred_contribs`) for the tree models; the LSTM is not explained (*as built*)	TreeSHAP is exact for tree ensembles; SHAP on an LSTM is costly on CPU
Experiment tracking	Tables in the main database plus JSON run reports (*as built*; no MLflow)	MLflow server is an extra service; revisit if needed
Background jobs	Job table plus in-process thread pool (*as built*; Celery/Redis not implemented)	Avoids Redis on an 8 GB laptop (NFR-03); consequence: one API process (`docs/13_limitations.md`)
Tests	pytest, Vitest, Playwright	Per spec
CI/CD	GitHub Actions	Per spec

3. Repository layout

```
EnergiQ-AI/
  frontend/            React + TypeScript app
  backend/app/         FastAPI app: api, models, schemas, security,
services; backend/migrations (Alembic)
  ml_engine/
    data/              ingestion, validation, resampling, quality
scoring
    features/          calendar, lag and rolling features
```

```
  forecasting/       baselines, Random Forest, XGBoost, LSTM
  anomaly/           Isolation Forest, rolling statistics
  explainability/    SHAP and importance
  evaluation/        metrics, backtests, plots, reports
data/raw|processed|sample   raw data is never committed
artifacts/models|reports    trained models; reports and profiles
tests/unit|integration|e2e
docs/  scripts/  docker/  infrastructure/aws  .github/workflows/
```

4. ML data flow

1. Ingest: chunked read, parse timestamps, map the missing markers (`?` and empty), sort, de-duplicate, quality score.
2. Resample to hourly mean, keeping only hours with enough valid minutes.
3. Chronological split; fit scalers on training data only.
4. Features: calendar, lags, rolling statistics (shifted so no future value is used).
5. Train and compare baselines, Random Forest, XGBoost and LSTM per horizon.
6. Evaluate on the held-out test split only; save metrics, plots, model artifact, data hash and config as a versioned run.
7. Score anomalies and compute explanations from the saved model.

5. Backend modules and endpoints (as built: `docs/api/API_REFERENCE.md`)

Area	Endpoints
Auth	register, login, current user
Datasets	upload, list, detail, quality report, readings, delete
Forecasts	create run, list, detail, compare
Anomalies	run detection, list with severity, acknowledge
Explainability	global importance, local explanation for a prediction
Optimisation	tariffs (CRUD), cost estimate, recommendations
Reports	downloadable HTML report per dataset; CSV export of a forecast
Models	list versions, evaluation results, activate, delete, monitoring (drift, realised error)
Admin	users, audit events
Health	liveness and readiness

6. Database entities (as built: 12 tables, `docs/diagrams/er_diagram.png`)

users, datasets, readings, jobs, model_versions, forecasts, anomaly_runs, anomalies, tariffs, recommendation_snapshots, notifications, audit_events. Roles are a column on `users`; dataset metadata, ingestion reports and evaluation results are JSON columns on their parent rows. Alembic migration `0001` (checked against the models on SQLite and PostgreSQL), with indexes on owner for every user-scoped table and on (dataset, timestamp) for readings and anomalies.

Energy readings are stored at the hourly canonical resolution; the raw 1-minute file stays on disk, not in the database, to keep SQLite small.

7. Security approach (as built: `docs/10_security.md`)

Per-user ownership checks on every dataset-scoped query; upload size and type limits with content validation; parameterised queries via SQLAlchemy; CORS restricted by environment; rate limiting on auth and upload routes; secrets from environment variables only; audit events for logins, uploads and deletions. Dependency vulnerability scanning in CI.

8. Deployment status ladder

Level	Status
Implemented locally	Yes: ML engine, API, web UI; tested on Python 3.11 with SQLite and PostgreSQL 16
Docker-tested	**No**: Dockerfiles and compose file written; images not built here (base-image registry unreachable from the workspace)
Cloud-configured	Yes: CloudFormation templates (cfn-lint clean) and GitHub Actions workflows; never executed
Actually deployed	**No**
Independently verified	**No**

No cloud resources were created. Details: `docs/12_deployment.md`.

9. Key decisions and open questions

Decision	Status
Primary target is `Global_active_power`, aggregated hourly	Confirmed in Phase 2
Hour is valid with at least 45 valid minutes	Confirmed in Phase 2
Test split 2010-01-01 to 2010-11-26	Confirmed in Phase 2
Validation: expanding-window time-series CV instead of one July–December block	Adopted (8 quarterly folds, `docs/05_ml_methodology.md`)
Celery and Redis deferred	Not implemented; in-process job pool used (limitation recorded)
Public-holiday calendar feature	Not implemented (no documented source); listed as future work
LSTM explanation method	Decided: not explained; XGBoost is the explained production model

Chapter 8. Machine-learning methodology

Citation numbers in square brackets refer to `docs/references.md`. Every number in this document is read from a file under `artifacts/reports/` that a script in `scripts/` produced.

1. Problem statement for the models

Predict the **hourly mean of `Global_active_power` (kW)** of one household for horizons of 1, 6 and 24 hours ahead, using only information available at the forecast origin.

- Data: UCI “Individual household electric power consumption” [17], 1-minute readings from December 2006 to November 2010 (CC BY 4.0).
- Target: the mean of the minute readings in each clock hour. An hour is **valid** only if at least 45 of its 60 minutes are valid. Hours with fewer valid minutes stay missing; **nothing is imputed**. 34,151 hours are valid (data-quality score 98.75, see `docs/04_eda_report.md`).
- The unit is kW (power), not kWh. A mean of 1.0 kW over one hour equals 1.0 kWh of energy; cost calculations use that identity and say so.

2. Forecasting framework

Direct multi-horizon forecasting. One model per horizon h is trained to predict the value at time t+h from features known at time t. This avoids feeding a model its own predictions (no error accumulation) and lets each horizon use the features that matter for it. The application bundles 24 XGBoost models (h = 1…24) so that a 24-hour curve can be produced; the published benchmark uses h = 1, 6 and 24.

2.1 Features (all built from data at or before the origin)

Group	Features
Recent consumption	`lag_0, lag_1, lag_2, lag_3, lag_5, lag_11, lag_23, lag_47` (value k hours before the origin)
Same time yesterday / last week	`seasonal_24`, `seasonal_168` aligned to the **target** time (only allowed when h ≤ 24, enforced in code)
Recent averages and variability	`roll_mean_24, roll_std_24, roll_mean_168, roll_std_168`
Time of day	`hour, hour_sin, hour_cos` of the target time
Day of week	`dow, is_weekend, dow_sin, dow_cos`
Season	`month, month_sin, month_cos`

Calendar features describe the **target** time, not the origin. Missing lags are left as NaN: XGBoost handles NaN natively, and Random Forest rows with missing inputs are excluded from scoring on the common evaluation mask (see §4). No public-holiday feature is used because the dataset has no declared country of residence beyond "near Paris"; holiday effects are listed as future work.

2.2 Chronological splitting and leakage control

- All splits are by time. There is **no shuffling anywhere**.
- Model selection uses an **expanding-window cross-validation** with 8 quarterly folds from 2008-Q1 to 2009-Q4. Each fold trains on everything before the fold start and is scored on the fold.
- The **test period, 2010-01-01 to 2010-11-26, is scored once**, after all choices (model families, hyper-parameters, the "best baseline by CV", the LSTM epoch count and seed) are fixed. Scaling parameters, fill values and the LSTM's early-stopping set come only from data before the cut-off.
- Unit tests prove the key properties on synthetic data (`tests/unit/test_features_splits.py`, `test_forecasting.py`): features use only past values, training never sees data after the cut-off, and an end-to-end API test (`test_forecast_uses_only_data_up_to_the_origin`) changes the future and confirms the forecast does not move.

3. Models

Model	Role	Notes
Naive (last hour)	baseline	Predicts the origin value. At h = 24 it is identical to seasonal-naive-24 (the origin is exactly 24 h before the target), so the two rows in the h = 24 table are the same numbers.
Seasonal naive 24 h / 168 h	baselines	Value at the same hour yesterday / last week
Random Forest [1]	learned model	scikit-learn; 2 candidate settings tried in CV
XGBoost [2]	learned model	3 candidate settings tried in CV; the application's production model
LSTM [3]	learned model	PyTorch, CPU only: hidden size 32, look-back 168 h, one network with three outputs (h = 1, 6, 24). Training scales with train-only mean/std, fills gaps with train-only hour-of-week means plus a missing-flag input, stops early on a validation block, then **refits on the full pre-test history for the selected number of epochs**. Seeds 42, 43, 44; seed 42 was declared the headline run before test scores existed.

Hyper-parameter search is deliberately small (the machine has 8 GB RAM and no GPU). The selected settings per horizon are stored in `artifacts/reports/benchmark/cv_results.json`: Random Forest `n_estimators=150, min_samples_leaf=10, max_features=0.5`; XGBoost `n_estimators=400, max_depth=6, learning_rate=0.03` for h = 6 and `n_estimators=300, max_depth=4, learning_rate=0.05` for h = 1 and 24 (the latter is also the single fixed configuration used by the application). The selection depends on the XGBoost version; see `docs/06_model_evaluation.md` §8.

The strongest baseline is chosen **by cross-validation, not by test score**; the evaluation also reports the comparison against the best baseline in hindsight, which is stricter.

4. Evaluation protocol

- **Common evaluation mask.** For each horizon every model is scored on exactly the same rows: those whose target is valid and whose inputs are available to *all* models. 7,233 hours are scored per horizon; about 686–687 of the 7,919–7,920 test hours are excluded for a missing target or missing inputs. A model never gets an easier subset than another.
- **Metrics.** MAE and RMSE in kW, MAPE and sMAPE in % (MAPE excludes hours with actual < 0.05 kW; none occur in the test rows), and R^2. MAE is the headline metric because it is in the unit of interest and robust to the heavy right tail of household load; the choice follows [7].
- **Uncertainty.** The paired difference of MAE between two models is estimated with a **day-block bootstrap** (resampling whole days, 312 days in the test span) to respect within-day autocorrelation; 95 % percentile intervals are reported. A difference whose interval excludes zero is called significant here; otherwise the models are called indistinguishable on this sample.
- **No claim beyond the data.** One household, one location, ~11 months of test data: the results say nothing about other households or years.

5. Reproducing the benchmark

```
python scripts/reproduce_benchmark.py --input
data/raw/uci_household_power/household_power_consumption.txt
```

runs the five steps in order; each is also a script of its own (`--input` is the raw file, `--help` shows the options):

1. `scripts/run_benchmark.py`: features, cross-validation, baselines, Random Forest and XGBoost, written to `artifacts/reports/benchmark/`.
2. `scripts/run_lstm.py --seeds 42 43 44`: the LSTM with three seeds, written to `artifacts/reports/benchmark/`.
3. `scripts/evaluate_models.py`: test metrics, bootstrap intervals and figures, written to `artifacts/reports/evaluation/`.
4. `scripts/run_explain.py`: TreeSHAP summaries, tariff cost and recommendations, written to `artifacts/reports/explain/`.
5. `scripts/run_anomaly.py`: anomaly summary and synthetic injection check, written to `artifacts/reports/anomaly/`.

The raw UCI file is not committed (`data/raw/` is git-ignored); download it from the DOI in [17] and place `household_power_consumption.txt` there. Use an interpreter built from `requirements.txt`: **XGBoost results depend on the XGBoost version** (`docs/06_model_evaluation.md` §8), and every output file records the versions that

produced it. On the development machine (2 CPU cores) the tree benchmark took 743 s and the LSTM with three seeds 667 s in the final run (`runtime_seconds` in the result files) (both while other jobs were also using the machine; an earlier solo run of the tree benchmark took 455 s); the full chain took about 25 minutes.

Chapter 9. Forecast evaluation

Source of every number: `artifacts/reports/evaluation/evaluation.json` and `model_comparison.md` (produced by `scripts/evaluate_models.py`). Test period: **2010-01-01 to 2010-11-26**, scored once. Protocol and definitions: `docs/05_ml_methodology.md`. Units are kW; the mean load in the scored rows is 1.055 kW.

1. Headline table (test MAE, kW; lower is better)

Horizon	Best baseline chosen by CV	Random Forest	XGBoost	LSTM (seed 42)
1 h	Naive (last hour) 0.4080	**0.3334**	0.3357	0.3533
6 h	Seasonal naive 168 h 0.5767	0.4354	**0.4327**	0.4383
24 h	Seasonal naive 168 h 0.5767	0.4415	**0.4413**	0.4414

n = 7,233 scored hours per horizon, identical rows for every model. Bold marks the lowest value in a row; differences of a few thousandths of a kW between the three learned models are within the noise discussed in §2 and §8.

Reduction of MAE relative to the CV-chosen baseline, computed from the table above:

Horizon	Random Forest	XGBoost	LSTM
1 h	18.3 %	17.7 %	13.4 %
6 h	24.5 %	25.0 %	24.0 %
24 h	23.4 %	23.5 %	23.5 %

Against the **strongest baseline in hindsight** (a stricter reference that uses test information to pick the baseline) XGBoost's reduction is 17.7 % at 1 h, 22.1 % at 6 h and 20.6 % at 24 h.

Full metrics (RMSE, MAPE, sMAPE, R^2) for all six methods are in `model_comparison.md`. For XGBoost: RMSE 0.486 / 0.597 / 0.604 kW and R^2 0.611 / 0.412 / 0.398 at 1 / 6 / 24 h.

2. What the statistics support

Paired day-block bootstrap of the MAE difference (95 % interval, 312 test days):

Comparison	h = 1	h = 6	h = 24
XGBoost − best CV baseline	−0.0723 [−0.0820, −0.0626]	−0.1440 [−0.1627, −0.1252]	−0.1354 [−0.1538, −0.1184]
Random Forest − best CV baseline	−0.0746 [−0.0838, −0.0650]	−0.1413 [−0.1598, −0.1235]	−0.1352 [−0.1533, −0.1187]
LSTM − best CV baseline	−0.0547 [−0.0639, −0.0453]	−0.1384 [−0.1590, −0.1182]	−0.1353 [−0.1560, −0.1155]
XGBoost − Random Forest	**+0.0022 [+0.0002, +0.0045]**	−0.0027 [−0.0065, +0.0009]	−0.0002 [−0.0038, +0.0034]
LSTM − XGBoost	**+0.0176 [+0.0134, +0.0218]**	+0.0056 [−0.0018, +0.0129]	+0.0001 [−0.0083, +0.0077]

Conclusions that the data supports:

1. **All three learned models beat the best baseline at every horizon**; every interval excludes zero.

2. **XGBoost and Random Forest are practically equivalent.** At 6 h and 24 h their paired intervals include zero. At 1 h Random Forest is better by 0.0022 kW [+0.0002, +0.0045]: the interval only just excludes zero and the gap is 0.7 % of the error, smaller than the shift caused by changing the XGBoost library version (§8). XGBoost is the one deployed in the application because it supports exact TreeSHAP and handles missing inputs natively, not because it is more accurate.
3. **At 1 h the LSTM is worse than XGBoost** by about 0.018 kW (interval excludes zero). **At 6 h and 24 h the LSTM and XGBoost are statistically indistinguishable.** The deep model gives no demonstrated benefit on this dataset, in line with the tabular-data findings in [10].
4. The 168-h seasonal baseline is *not* clearly different from the 24-h one at 6 h (−0.0209 [−0.0473, +0.0062]), so the "best baseline" choice matters little there.

3. Seed robustness of the LSTM

Seed	Selected epochs	MAE h = 1	MAE h = 6	MAE h = 24
42 (headline)	11	0.3533	0.4383	0.4414
43	6	0.3623	0.4510	0.4512
44	8	0.3512	0.4405	0.4470

Seed-to-seed spread (≈0.01 kW) is about half the 1-h gap to XGBoost and comparable to the differences between models at 6 and 24 h, so no ranking of the LSTM against the tree models at those horizons should be read into a single seed.

4. Honest reading of the error level

- The MAE of 0.33–0.44 kW is **32–42 % of the mean load**. Household load is spiky (cooking, heating, appliances) and a single-meter series carries little information about what the occupants will do. R^2 is 0.61 at 1 h but only **about 0.40 at 6 h and 24 h**: the models explain a minority of the variance at longer horizons.
- The improvement over simple baselines is real but modest in absolute terms (≈0.08–0.14 kW MAE).
- MAPE is large (42–62 %) because hourly values near zero inflate percentage errors; it is reported for completeness, not as the headline.
- The forecast range shown in the application is an **80 % band built from the 10th and 90th percentiles of each horizon's residuals on the application's own chronological hold-out block** (`residual_q10_kw`, `residual_q90_kw`). It is an empirical, horizon-wise band, not a validated prediction interval: its coverage was not measured on the 2010 benchmark test period, and it assumes the error distribution does not change between the hold-out block and the forecast date.

5. Application models versus the benchmark

The numbers above are the **research benchmark** (full UCI data, tuned configurations, 2010 test period). The **application** trains its own models per dataset with a fixed XGBoost configuration, evaluates them on the last 20 % of that dataset's timeline, and refits on all data for deployment (`docs/09_backend_and_api.md` §5). On the bundled one-year sample (8,760 hourly rows, 2009) the application reported hold-out MAE of 0.392 / 0.516 / 0.513 kW at 1 / 6 / 24 h (measured in a real end-to-end run; training took 12 to 25 s on the development machine). These figures use a different period and much less history than the benchmark and **must not be compared with it**: a one-year dataset shows each calendar month only once, so yearly seasonality cannot be validated across years (the application attaches a caveat to the model only when the data spans fewer than 360 days, so this one-year sample carries no warning; treat its season-dependent forecasts with that in mind).

6. Figures

All in `artifacts/reports/evaluation/`: `fig_mae_by_horizon.png`, `fig_actual_vs_predicted_week.png`, `fig_error_by_hour.png`, `fig_error_by_month.png`, `fig_residuals.png`, `fig_actual_vs_predicted_scatter.png`. The web application serves the same images through the authenticated Benchmark page.

7. Threats to validity

- One household, one site, one test year: results do not transfer to other homes without re-evaluation.
- The 11-month test period does not include a full second winter; seasonal behaviour in the test period is inferred from earlier years.
- Hyper-parameter search was intentionally small; a larger search could change absolute numbers, though the CV differences between candidates were small.
- The clock-change check (`docs/04_eda_report.md`) found the data to be in a single local-time convention; if that is wrong for some hours, calendar features are slightly off for those hours.

8. Reproducibility and library versions

Every result file written by the research scripts records the interpreter and library versions that produced it (`environment` in `cv_results.json`, `lstm_results.json`, `evaluation.json`, `shap_global.json`, `anomaly_summary.json`). All numbers in this chapter come from one run of `scripts/reproduce_benchmark.py` under the **pinned** environment of `requirements.txt` (Python 3.11.15, numpy 2.4.6, pandas 3.1.0, scikit-learn 1.9.1, xgboost 3.2.0, torch 2.15.0, CPU only, 2 threads).

A correction made during final verification. The first version of these results had been produced earlier in the project, before the environment was pinned and recorded. Re-running the whole chain from the raw file under the pinned environment showed:

- The naive and seasonal baselines, Random Forest (to about 10^{-15} kW) and the LSTM (all three seeds: same selected epochs, same validation history, same test metrics) **reproduced exactly**.
- **XGBoost did not**: its predictions depend on the XGBoost build. Test MAE moved from 0.3321 / 0.4354 / 0.4432 kW to 0.3357 / 0.4327 / 0.4413 kW at 1 / 6 / 24 h (at most 0.0036 kW), the cross-validation chose a different configuration for h = 1 (300 trees, depth 4, learning rate 0.05, instead of 400 / 6 / 0.03), and the 1-hour XGBoost–Random Forest comparison changed from "indistinguishable" to "Random Forest marginally better". The conclusions that matter did not change: all learned models beat the baseline at every horizon, XGBoost and Random Forest are within 0.003 kW of each other, and the LSTM gives no demonstrated benefit. The documents, figures and SHAP numbers were regenerated and updated to the pinned-environment values; the superseded numbers remain in the Git history.
- The anomaly results (events, summary, synthetic injection check) and the recommendation example were **identical** after the re-run, because they do not involve XGBoost.
- The same effect shows in the application: on the bundled sample, the hold-out MAE at 1 / 6 / 24 h is 0.3915 / 0.5157 / 0.5128 kW with xgboost 3.2.0 (pinned; identical on repeated runs and with 1 or 2 threads) and 0.3895 / 0.5117 / 0.5171 kW with xgboost 3.4.1 (the system Python 3.13 of the build workspace).

So: results are **bit-reproducible on the same library versions** and move by up to a few thousandths of a kW when only the XGBoost version changes. Differences of that size between models should never be read as a ranking. Reproduction from raw data took about 25 minutes on 2 CPU cores (peak memory: 600 MB for the tree benchmark, 1.1 GB for one LSTM seed; measured, see `docs/11_testing.md`).

Chapter 10. Explainability, cost and recommendations

Numbers come from `artifacts/reports/explain/` (produced by `scripts/run_explain.py`) unless stated otherwise. Code: `ml_engine/explainability/shap_tree.py`, `ml_engine/optimization/`. Tests: `tests/unit/test_explain_optimize.py`.

1. Explainability

1.1 Method

The deployed forecasters are XGBoost tree ensembles, for which **exact TreeSHAP** values can be computed [5, 6]. They are obtained from XGBoost's own `pred_contribs` output, so no sampling or approximation is involved. For a prediction, the SHAP values of all features plus the base value add up to the model output (in kW).

- **Additivity is verified, not assumed.** On the 7,233 benchmark test rows the largest absolute gap between "base value + sum of contributions" and the model prediction was **2.6×10^{-6} kW (h = 1), 3.8×10^{-6} kW (h = 6) and 2.9×10^{-6} kW (h = 24)** (`shap_global.json`, `max_additivity_error_kw`). The unit tests assert the same property.
- **Feature groups.** The 24 engineered features are shown to users in six plain-language groups (Recent consumption, Same time yesterday / last week, Recent averages and variability, Time of day, Day of week, Season), because correlated features such as `hour`, `hour_sin` and `hour_cos` split credit between them and are misleading when read one by one.
- The LSTM is **not** explained: SHAP for a sequence model on a CPU-only machine is expensive and the LSTM was not the best model. The application explains the XGBoost models only, and the interface says so.

1.2 What the models rely on (share of mean |SHAP|, benchmark test rows)

Group	h = 1	h = 6	h = 24
Recent consumption	**64.0 %**	15.4 %	**24.5 %**
Time of day	17.1 %	**25.7 %**	23.2 %
Same time yesterday / last week	7.2 %	24.6 %	16.7 %
Recent averages and variability	4.5 %	14.2 %	12.3 %
Season	3.6 %	11.1 %	13.5 %
Day of week	3.6 %	9.1 %	9.8 %

Reading: one hour ahead the model mostly extends the latest reading (`lag_0` alone carries 47.3 % of the attribution). At 6 hours the daily rhythm (time of day, 25.7 %) and the same-time-yesterday / last-week values (24.6 %) matter most, and `lag_0` is no longer among the ten most important features. At 24 hours the shares are spread out: the group of recent-consumption lags is still the largest (24.5 %) but `lag_0` alone is only 7.0 %, and the single most important feature is the hour of day (20.6 %). This is consistent with the autocorrelation analysis in

`docs/04_eda_report.md`. With groups of correlated features, the ranking of two groups that are a few points apart (for example time of day against recent consumption at 24 h) should not be over-interpreted.

These shares were regenerated under the pinned library versions (`docs/06_model_evaluation.md` §8); an earlier run with a different XGBoost build gave similar but not identical values (for example 61.4 % instead of 64.0 % for recent consumption at 1 h).

Caveat. SHAP describes what the *model* uses, not what *causes* household consumption. A large "time of day" share does not mean the clock changes consumption; it means the model found the daily rhythm predictive.

1.3 Local explanations

`POST /api/v1/explain/local` returns, for one forecast origin and horizon, the base value, the contribution of each feature group in kW, the top individual features and a one-sentence narrative generated from those numbers (`narrative()` in `shap_tree.py`). Nothing is generated by a language model; the sentence is a template filled with the computed contributions.

2. Tariff-based cost

- A `Tariff` has a currency, 24 weekday and 24 weekend prices per kWh and an optional fixed daily charge. Prices must be finite and non-negative (validated).
- Cost of a series = Σ over valid hours of (mean kW × 1 h) × price of that hour (+ fixed charge × days). **Missing hours are not costed**; the response reports the number of valid hours so the reader can see what the figure covers.
- The built-in **DEMO tariff** (`DEMO_TARIFF`) uses illustrative prices and is labelled "DEMO tariff (not a real utility price)" everywhere, including the API output (`is_demo_tariff: true`) and the user interface. It exists so the feature can be tried; real costs require the user's own tariff.
- Example from the saved run (`recommendations.json`): the last 86 days with data (2010-08-28 to 2010-11-26, 2,072 valid hours) used 2,290.6 kWh, which the DEMO tariff prices at 462.28 EUR. This is an arithmetic illustration of the tariff, **not a statement about any real bill**.

3. Recommendations

Rule-based and transparent. Each recommendation carries its **evidence** (the numbers that triggered it) and, where it quantifies a saving, the **assumption** it rests on.

Rule	Fires when	Output
`review_anomalies`	High-severity anomaly events exist in the analysed window	Pointer to the events; no saving claimed
`reduce_baseload`	Always-on load (a low quantile of hourly load) is a material share of energy	Scenario: "if always-on load is reduced by 10 %" – kWh and cost per 30 days
`shift_peak_load`	Energy is concentrated in the most expensive tariff hours (concentration ≥ 1.2 × its time share)	Scenario: "10 % of the dearest-period energy is moved to the cheapest hours"
`forecast_peak`	The next-24-hour forecast peak reaches a high quantile of recent history (priority is raised if it falls in the most expensive tariff period)	Time and size of the forecast peak; no saving claimed
`data_gaps`	Less than 90 % of the last 7 days contain readings	Warning that forecasts and anomaly checks are less reliable

For the saved example window the engine produced three recommendations: review anomalies; check always-on devices (scenario estimate 21.3 kWh and 4.29 EUR per 30 days under the DEMO tariff); and move flexible use out of the most expensive hours (scenario estimate 13.1 kWh and 2.36 EUR per 30 days). **These are scenario estimates under stated assumptions, not forecasts of savings**; the interface labels them as such. The 10 % figures are the engine's defaults, chosen for illustration rather than learned from the household.

3.1 Cheapest-window scheduler

`POST /api/v1/optimize/schedule` finds the start hour within a 24-hour window (beginning at the requested time, or just after the last data point) that minimises the tariff cost of running a device of a given power (≤ 1000 kW) for a given duration (1–24 h). It is an exhaustive search over at most 24 start hours, so the result is optimal for the stated tariff. When a forecast is available, the total forecast load during the run is added as a very small tie-breaking term (weight 10^{-3} per kW-hour), so it effectively only decides between windows whose costs are equal or differ by a negligible amount (a few thousandths of a currency unit), preferring the one that overlaps less forecast load. Remaining ties go to the earliest start. It schedules a device the user says they can move; it does not know whether they actually can.

4. Limitations specific to this module

- Explanations are for one model family and one household's data.
- Tariffs are user-entered; no live market prices are used.
- The recommendation thresholds are heuristics with fixed defaults; they are not validated against measured behaviour change.

Chapter 11. Anomaly detection

Numbers come from `artifacts/reports/anomaly/` (`scripts/run_anomaly.py`, UCI dataset, 34,151 valid hours). Code: `ml_engine/anomaly/detector.py`. Tests: `tests/unit/test_anomaly.py` (10 tests) and `tests/integration/test_api_ml.py`.

1. The central limitation: there are no labels

The dataset does not say which hours are faulty or unusual. **No precision, recall, F1 or "accuracy" can be computed on the real data, and none is reported.** What this module offers is a transparent statistical definition of "unusual for this household at this time", an explanation for every flag, and a *synthetic* sanity check that is clearly labelled as synthetic (§5). Flags are leads for a person to look at; they are not confirmed faults.

2. Method

For every valid hour the detector forms a **causal expectation** and measures how far the reading is from it. "Causal" means the expectation for hour t uses only data before t, so the same function works on live data.

1. **Expected value.** Median of the same hour-of-week over the previous 8 weeks (at least 4 required). The median ignores earlier spikes.
2. **Residual scale by hour of day.** The empirical distribution of (actual − expected) is built per hour of day over the trailing 26 weeks (at least 56 days), so the 03:00 band is narrow and the 19:00 band is wide. Hours with too little history are not scored.
3. **Alert band.** An hour is a *deviation* candidate if the residual lies beyond the 0.995 empirical quantile of its hour-of-day (upper tail for spikes, lower tail, 0.005, for drops)

and the absolute deviation is at least 0.5 kW. The second condition removes statistically unusual but practically irrelevant wiggles at low load.

4. **Flat-run rule.** Four or more consecutive hours with an identical hourly value are flagged as a possible stuck sensor or missing-data placeholder. The threshold of 4 was chosen from the empirical run-length distribution of the same dataset, so it is *not* independent evidence.
5. **Isolation Forest corroboration** [4]. An Isolation Forest (150 trees) is fitted on the first 60 % of the series using seven per-hour features (the value, its change from the previous hour, its ratio to the previous-24-hour mean, the band residual, sine/cosine of the hour and a weekend flag); hours it scores above its 0.995 quantile **and** with a residual $z \geq 2.5$ are flagged as "unusual pattern". The first 60 % are in-sample for that model (reported in the output). It is used to add pattern-type flags, not to override the band.
6. **Events and severity.** Flagged hours separated by at most one hour are merged into events. Severity is set from the excess ratio of the worst hour (`_severity_from_excess`: ≥ 2.5 high, ≥ 1.5 medium, otherwise low). Each event carries an auto-generated plain-language explanation built from its numbers.

Sensitivity in the application maps to the band quantile: low = 0.999, medium = 0.995 (default), high = 0.99.

Why not a simple z-score?

The first implementation used a MAD z-score; it flagged 12.4 % of all hours and still missed a known 401-minute flat period found in Phase 1. It was replaced by the quantile band above, which respects the heavy, hour-dependent tail of household load. This iteration was guided by the same dataset, which is a form of tuning on the data and is disclosed here.

3. Results on the real dataset (medium sensitivity)

Item	Value
Valid hours scored by a baseline	32,572 of 34,151 valid hours
Hours flagged	695 (2.04 % of valid hours)
…by deviation band	651
…by flat-run rule	5
…by Isolation Forest only (with $z \geq 2.5$)	39
Events after merging	484 (376 low, 89 medium, 19 high)
Event types	263 spikes, 186 drops, 36 unusual patterns, 1 flat run

Cross-check against two irregularities noticed during EDA, *before* the detector was built (these are consistency checks, not accuracy measurements):

- A 401-minute run at a constant 0.360 kW on 2008-10-06 (the 8 hours from 10:00 to 17:00 are touched): **5 of those 8 hours flagged**, all by the flat-run rule.
- A 10-day low-use block in August 2008 (240 hours): **0 hours flagged**. The detector treats a sustained low-use period as the household's new normal after a few days and does not call it an anomaly. Whether that period is a holiday absence or a fault cannot be determined from this data.

The highest-severity events in the saved output are isolated drops (for example 2010-02-19 02:00: 0.31 kW against 0.81 kW expected) and a 12-hour drop on 2009-02-21/22; see `events.csv` and `fig_anomaly_examples.png`.

4. Behaviour you should expect

- About 2 % of hours are flagged at the default setting by construction (a 0.995 band on each tail plus extra rules); this is a **chosen alert rate**, not a measured false-alarm rate.
- Slow changes (new appliance, season change) are absorbed after a few weeks because the baseline is a rolling median of recent weeks.
- The first weeks of any dataset cannot be scored (no baseline yet).
- The first run on a new dataset needs at least 12 weeks of valid history; the API refuses shorter data with an explanatory message.

5. Synthetic injection check (clearly synthetic)

To see whether the detector responds to anomalies of known size, 60 events of each kind were injected into a copy of the real series at random positions and the detector was run again (`injection_check.json`). Because the underlying series already contains genuine irregularities and because the injected shapes are my own choice, the numbers describe sensitivity to these particular synthetic patterns only:

Injected pattern	Recall on injected hours	Recall when expected load ≥ 1 kW	Flag rate on untouched hours
Spike to ×3 expected	61.7 %	75.0 %	1.8 %
Spike to ×1.6 expected	31.7 %	35.0 %	1.9 %
Drop to 5 % of expected	10.0 %	30.0 %	2.0 %
8-hour flat run	100 %	100 %	1.9 %

Read this honestly: large spikes and flat runs are detected well; **moderate deviations and drops are often missed** (a drop at an hour when the expected load is already low cannot exceed the 0.5 kW minimum deviation). The detector is deliberately conservative. These percentages are not an accuracy figure for real faults.

6. In the application

`POST /api/v1/datasets/{id}/anomalies/run` starts a background job; `GET /api/v1/anomalies` lists events with filters (severity, acknowledged, date range); `PATCH /api/v1/anomalies/{id}` acknowledges one; `GET /api/v1/anomalies/summary` returns counts. Only the **latest run per dataset is kept**: starting a new run replaces the previous run's events, so acknowledgements made on the old events are not carried over (the acknowledgement actions themselves remain in the audit log).

Chapter 12. Backend design and API

Code: `backend/app/`. Full endpoint list with access rules: `docs/api/API_REFERENCE.md` (generated from the running application by `scripts/make_api_reference.py`); schema: `docs/api/openapi.json`. Diagrams: `docs/diagrams/`.

1. Layers

Layer	Files	Responsibility
Entry point	`main.py`	`create_app(settings)` factory: CORS, request-id / security-header / metrics middleware, routers, health and `/metrics`, start-up job recovery, optional administrator seeding

Layer	Files	Responsibility
HTTP API	`api/auth, datasets, ml, anomalies, optimize, reports, admin`	Validation, authorisation, ownership checks, status codes. No ML code.
Services	`services/core.py, data.py, ml.py`	Audit trail, notifications, job runner and metrics (`core`); upload streaming, ingestion into the database, series loading (`data`); training, forecasting, explanation, anomaly job, monitoring (`ml`)
ML engine	`ml_engine/` (no web dependencies)	Ingestion/quality, features, models, evaluation, anomaly detection, SHAP, tariff and recommendations
Persistence	`models.py, db.py, migrations/`	SQLAlchemy 2 models; SQLite (WAL, foreign keys on) for development and tests, PostgreSQL 16 in production; Alembic migration `0001`
Security	`security.py, config.py`	Password hashing, JWT, rate limiter, roles, settings and production safety checks

Settings come from environment variables with prefix `ENERGIQ_` (or `.env`); a production start-up refuses the built-in development secret, secrets shorter than 32 characters and wildcard CORS.

2. Data model (`docs/diagrams/er_diagram.png`)

Twelve tables, generated into the diagram from the ORM so it cannot drift: `users`, `datasets`, `readings`, `jobs`, `model_versions`, `forecasts`, `anomaly_runs`, `anomalies`, `tariffs`, `recommendation_snapshots`, `notifications`, `audit_events`. Every user-owned row carries `owner_id`; foreign keys cascade on delete from dataset to its readings, models, forecasts and anomalies. The Alembic migration is checked against the models by a test (`test_migrations.py`), on SQLite and, where available, PostgreSQL.

3. Background jobs

Upload processing, training and anomaly detection can take seconds to minutes, so each is a **job**: the request returns `202` with a job id, a worker thread runs it, and its state (`queued → running → succeeded | failed`, progress 0–1, message, result, error) lives in the `jobs` table. Properties worth knowing:

- Job rows survive a restart; on start-up any job left `queued` or `running` by a previous process is marked `failed: Interrupted by a server restart` (it can never finish).
- Only expected user-facing errors (bad data, missing file) reach the job's `error` field; unexpected exceptions are logged with a stack trace and shown to the user as "Internal error; see server logs" (tested).
- A second train/anomaly job for the same dataset is refused with 409 while one is running.
- The pool lives inside the API process. Therefore **run exactly one API process per deployment** (see `docs/13_limitations.md`).

4. Upload pipeline

`POST /api/v1/datasets` (multipart): extension whitelist (`.csv .txt .tsv`) → streamed to a uuid-named temporary file with a size cap (default 50 MB in code, 200 MB in the compose file) so the client's file name never touches the file system → NUL-byte/binary rejection →

declared unit (`kW, W, MW, kWh, Wh, MWh`) and time-zone convention (`logger_clock, utc, local_with_dst`) validated → job. Parsing is tolerant of delimiter and encoding but strict about meaning: unparseable timestamps, non-numeric values and wrong headers are rejected with explanations, and the quality report shows completeness, duplicates, gaps and flat runs. Energy units are converted to power only when the sampling interval is known; the report states the assumption.

5. Forecast and explanation endpoints

- `POST /datasets/{id}/append` (file only; the column mapping stored with the dataset is reused) adds newer readings. Hours present in the new file with a value **replace** the stored value; hours that are absent from the new file, or present but blank, **keep the stored reading** (an earlier version erased stored readings wherever the new file had a gap; this was found while writing documentation and is covered by a regression test, `test_append_replaces_overlapping_values_but_never_erases_readings_with_gaps`, which fails on the old code). A model trained before an append stays active until the user retrains.
- `POST /datasets/{id}/train` works in two stages. (1) **Evaluation:** for h = 1, 6 and 24 an XGBoost model is fitted only on rows whose target lies before a chronological hold-out (the last 20 % of the dataset's timeline) and scored once on that block against the baselines; per-horizon residual quantiles and a SHAP summary are stored. (2) **Deployment:** the 24 models of the bundle (h = 1…24) are **refitted on all the data**, so the model used for forecasting has also seen the hold-out period. The reported hold-out metrics therefore describe the protocol, not the exact deployed models. The application uses one fixed, untuned XGBoost configuration (`n_estimators=300, max_depth=4, learning_rate=0.05`, the first entry of the benchmark's grid) so that training takes seconds to minutes on a laptop; it is not the per-horizon configuration chosen by cross-validation in the research benchmark. The model is optionally activated.
- `POST /forecasts` builds features from the data at the origin (default: last timestamp) and returns 24 points with an 80 % range. A regression test changes the data after the origin and checks the forecast is unchanged.
- `POST /explain/local` returns exact TreeSHAP contributions; `GET /explain/global` returns the stored benchmark SHAP summary; `GET /benchmark` and `/benchmark/figures/{group}/{name}` serve the stored benchmark results and a whitelist of figure files (path traversal is tested).

6. Observability endpoints

`/health/live` (process up), `/health/ready` (database reachable, otherwise 503), `/metrics` (Prometheus text format: request counts and latency histograms per route template and status, job counts by status), `/api/v1/admin/system` (admin only) and `/api/v1/models/{id}/monitoring` (drift and realised error, `docs/14_mlops_monitoring.md`). Every response carries an `X-Request-ID`.

Chapter 13. Web interface

Code: `frontend/` (about 6,700 lines of TypeScript/TSX in 58 source files). Run instructions: `frontend/README.md` and `docs/manuals/installation.md`. Usage: `docs/manuals/user_guide.md`. Screenshots: `docs/screenshots/` (taken from the real application against a real backend, Chromium only).

1. Stack and structure

React 18.3, TypeScript (strict), Vite 6, Tailwind 3, Recharts 2.15, React Router 6.30. Dependencies are pinned exactly in `package.json` and locked in `package-lock.json`; `npm ci` reproduces the install.

Folder	Content
`src/api`	One typed client for the API (`client.ts`; endpoint groups in `endpoints.ts`; response types in `types.ts`), job polling (`polling.ts`). Errors carry the server's message; HTTP 429 shows how long to wait (`Retry-After`).
`src/auth`	Auth context, route guards (sign-in required; admin page only for admins), `WriteOnly` (hides write actions from viewers).
`src/hooks`	`useApi` (loading/error/data with abort), `useJob` (poll a background job), `useBlobUrl` (authenticated image download), `useTheme`.
`src/pages`	Login/Register, Dashboard, Datasets and dataset detail, Forecast, Models, Anomalies, Explainability, Optimisation (tariffs, cost, recommendations, scheduler), Benchmark, Jobs, Notifications, Admin (users, audit, system), 404.
`src/components`	Charts (line/band/markers/waterfall with a table alternative for each), form fields, tables, notices, error boundary.
`src/lib`	Formatting, validation (same password rule as the server), safe internal links, storage wrapper, time handling.

Uploads, training, forecasting and detection are **background jobs**: the page shows a progress bar and updates when the job finishes, so the browser never waits on one long request.

2. Design decisions that matter

- **Real data only.** Every number, series and table row comes from an API response; there is no demo data in the interface. The one exception is the built-in DEMO tariff, which the *server* provides and which is labelled "not a real utility price" wherever it is shown (requirement NFR-09).
- **Honest wording** is part of the interface, not an afterthought: anomalies are "unusual hours, not confirmed faults" and no accuracy is shown for them; savings are labelled scenario estimates with their assumption; the hold-out period and the baseline comparison sit next to every accuracy figure; forecast bands are called a rough range; warnings and caveats sent by the API are displayed verbatim.
- **Token handling.** The access token lives in memory and `sessionStorage` (cleared on sign-out and on any 401). Only the theme preference uses `localStorage`; lint rules forbid direct `localStorage` use elsewhere. There is no `dangerouslySetInnerHTML` (also lint-banned), so server text is always escaped by React.
- **Authenticated downloads.** CSV exports, the HTML report and the benchmark PNGs are fetched with the bearer token as blobs (an `<img src>` cannot send a header), and the object URLs are revoked afterwards. Tokens are never put in URLs (the backend rejects that, see `docs/10_security.md`).
- **Open-redirect protection.** The post-login return path and links inside notifications pass through `safeInternalLink`, which refuses `//host`, backslash and off-site targets.
- **Roles.** Viewers see no write buttons and non-admins cannot open `/admin`. This is convenience only; the server enforces roles on every request (role-matrix test in `tests/integration/test_security.py`).
- **Accessibility.** Semantic landmarks and headings, labelled inputs, visible focus, a skip link, a **Chart/Table toggle on every chart**, status never conveyed by colour alone,

light/dark themes (system setting plus manual toggle), responsive down to phone width. Checked by keyboard and automated tests; **not** checked with a screen reader.

- **Charts** follow one rule set: one y-axis per chart, thin lines, a legend whenever there are two or more series, a light forecast band, event markers with a table of the same events.

3. Verification (what was run, with results)

Commands run in `frontend/` on Linux with Node 22 (re-run independently by the project lead after the frontend was written, from a clean `npm ci`):

Check	Result
`npm run lint`	exit 0, no errors or warnings
`npm run typecheck` (`tsc -b --noEmit`, strict)	exit 0
`npm test` (Vitest + Testing Library, jsdom)	**122 tests in 8 files, all passed**
`npm run build`	exit 0; largest chunk `vendor-recharts` 417 kB (113 kB gzip), `vendor-react` 167 kB (54 kB gzip)
`npm audit --omit=dev --audit-level=high` (production dependencies)	exit 0: **0 high/critical**, 2 moderate (react-router 6.x, see §4)

The 122 tests are split as pages 39, auth 18, library 19, API client 16, validation 11, components 8, job polling 5 and polling helper 6. They run against **45 JSON fixtures captured from the running backend** (only the JWT in the login fixtures was replaced by a placeholder), so the interface is tested against the real response shapes rather than shapes invented by the test author. They cover rendering from API data, client-side validation that sends nothing when invalid, error and 429 handling, the real route-guard flow, role gating, escaping, and storage decisions.

Live checks. The frontend author drove a production build in real Chromium (Playwright) against a real backend: registration and sign-in, sample load, upload, append and delete of a throwaway dataset, training with progress, forecast and look-back forecast, anomaly detection with acknowledge/unacknowledge, explainability, tariff create/delete, all 11 benchmark figures, jobs and notifications, administration (role change, audit filter, system stats), viewer read-only behaviour, authenticated downloads, theme toggle, the real 401 and 429 paths, and the dashboard figures; a final script passed 9 of 9 checks with an empty browser console. 33 page/theme/viewport combinations showed no horizontal overflow. The screenshots in `docs/screenshots/` come from these runs.

Browser end-to-end test in the repository (`tests/e2e/test_ui_workflow.py`): a real Chromium drives the built app against a real uvicorn server through registration (including the weak-password refusal), sample load, model training, forecast and anomaly detection, and asserts that the numbers on screen (dataset hours, 1-hour hold-out MAE, forecast peak) equal what the API returned, that no request failed with a 5xx status and that the console stayed free of errors. It skips with a stated reason when Node, the built bundle or Chromium is missing.

4. Known gaps and risks

- **Dependency advisories.** The production dependencies carry two *moderate* advisories in react-router 6.x (fixed only in 7.18 and later): an open redirect through a backslash in `<Link>`/`useNavigate`, and a constructor-injection issue in server-side-rendering hydration. The second does not apply to a client-only application. For the first, the application only navigates to targets it builds itself or that pass through `safeInternalLink` (which rejects `//`, backslash and off-site paths, covered by tests); the upgrade to react-router 7 was deferred as a deliberate change. The full `npm audit` (including development tools that never ship in the bundle) still lists **2 critical** advisories in `vitest`/`tinypool`; the fix is a breaking major upgrade of the test runner,

and the issues concern the test runner's own mock/worker options, not the application. One high advisory in `postcss` (build-time) was removed by updating to 8.5.32 during final verification. Recharts 2.x is flagged as deprecated by npm; it was kept for stability.

- Rendering was checked in **Chromium only**; other browsers, slow networks and screen readers were not tested.
- Chart geometry is verified only in a real browser; jsdom tests check data and table alternatives, not pixels.
- The anomaly list is limited to 500 events per request by the API and has no server-side date filter, so the date filter works on the loaded events only.
- Drift thresholds quoted in the monitoring text (PSI/KS) are copied from the backend documentation, not read from an endpoint; if the backend changes them the text goes stale.
- Frontend line/branch coverage was not measured in the final pass (`npm run test:coverage` exists).
- Models trained before the forecast-band sign fix (`residual_band`, found while writing the documentation) keep their stored quantiles, so their band is mirrored; **retrain** such models. No deployment of this project predates the fix.

Chapter 14. Security

Scope: the web application and API in this repository. Standards used as a checklist: OWASP Top 10 (2021) and OWASP API Security Top 10 (2023); neither is a certification and no external penetration test was performed. Tests: `tests/integration/test_security.py` (16 tests), `test_api_core.py`, `test_api_ml.py`, `test_migrations.py`.

1. Controls and where they are verified

Area	Control	Evidence
Passwords	argon2id (library defaults); 10–128 characters, letters and digits required, a short list of common passwords and trivially repetitive values refused; hashes never returned by any endpoint	`test_password_hashes_never_leave_the_server`, `test_api_core`
Login	Generic failure message; the same CPU time is spent for unknown e-mails as for wrong passwords (no timing oracle for account existence); 10 attempts/minute per IP and 5 per 5 minutes per IP+e-mail → 429 with `Retry-After`	`test_api_core` login tests
Tokens	HS256 JWT, `exp` and `sub` required, 60-minute lifetime, secret ≥ 32 random characters enforced in production; the user row is re-read on every request so a deactivated user or changed role takes effect immediately; token in the query string is rejected	`test_token_in_query_string_is_not_accepted`, tampering and expiry tests in `test_api_core`
Authentication coverage	Every `/api/v1` operation except register and login returns 401 without a token	`test_every_api_route_requires_authentication` (iterates the OpenAPI schema; ≥ 40 operations)
Authorisation	Roles `admin`, `analyst`, `viewer`. Viewer is read-only. Admin endpoints need admin.	`test_role_matrix_…` derives the expected access from the route dependencies and calls each restricted operation as each role
Object-level access	Every dataset, model, forecast, anomaly, tariff and notification lookup filters by `owner_id`; a foreign id returns **404**, not 403, so ids are not confirmable	`test_idor_on_tariffs_notifications_and_anomalies`, `test_api_core` ownership tests
Mass assignment	Registration ignores client-supplied	`test_mass_assignment_cannot_`

Area	Control	Evidence
	`role` / `is_active` (always `analyst`); role changes are admin-only and audited; an admin cannot demote or deactivate themselves	`set_role`, admin tests
Injection	Only parametrised SQLAlchemy queries; enum-like filters are regex-validated query parameters; a dataset named `x'; DROP TABLE datasets; --` is stored as plain text and the table survives	`test_injection_strings_are_data_not_code`
XSS	The API returns JSON with `default-src 'none'`; the downloadable HTML report escapes every user-controlled string and ships a CSP without scripts; the React UI never uses `dangerouslySetInnerHTML`	`test_report_escapes_user_text`
Uploads	Extension whitelist, streamed size cap, NUL/binary rejection, uuid file names, per-user storage directory, upload rate limit (30/hour)	`test_api_core` upload tests, `test_upload_rate_limit`
Path traversal	Benchmark figures are served from a fixed whitelist of group and file names	`test_api_ml` figure tests
Rate limiting	Registration 10/hour per IP, login as above, upload 30/hour per user	`test_register_rate_limit`, `test_upload_rate_limit`
Client IP	With `ENERGIQ_TRUST_PROXY=true`, the address is the entry *N* places from the **right** of `X-Forwarded-For` (N = `ENERGIQ_TRUSTED_PROXY_HOPS`, default 1). A client-forged left entry cannot dodge a limit. With the option off the header is ignored.	`test_forged_forwarded_for_…`, `test_client_ip_hops_unit`
Transport / headers	`X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, `Cache-Control: no-store` and a restrictive CSP on API responses, HSTS in production; nginx adds a CSP for the UI	`test_security_headers_and_cors`
CORS	Explicit origin list, no credentials, only needed methods and headers; a foreign origin gets no CORS headers; wildcard refused in production	`test_security_headers_and_cors`, `test_api_core`
Error hygiene	Unhandled exceptions return `{"detail":"Internal server error","request_id":…}` (the details are only in the server log); job errors are filtered the same way	`test_internal_errors_do_not_leak_details`, `test_job_errors_hide_internal_exceptions`
Secrets	No secrets in the repository; `.env` is git-ignored; compose refuses to start without `POSTGRES_PASSWORD` and `ENERGIQ_SECRET_KEY`; on AWS the database password is generated and managed by RDS/Secrets Manager and injected as a container secret	`.gitignore`, `docker-compose.yml`, `infrastructure/aws/energiq-stack.yaml`
Audit	Registration, login success/failure, uploads, deletions, role changes, training, anomaly runs, tariff changes are written to `audit_events` with user, IP and resource	`test_api_core`, `GET /api/v1/admin/audit`
Containers	Non-root user (uid 10001), no build tools in the runtime image, one process, health check, read-only application code	`docker/backend.Dockerfile` (**not built here**, see §3)
Cloud network	Database and file system in subnets with no internet route, reachable only from the API security group; load balancer drops invalid header fields; EFS and RDS encrypted	`infrastructure/aws/energiq-stack.yaml` (cfn-lint clean; **not deployed**)

2. Static and dependency scans (run in this project)

Tool	Scope	Result
`bandit -r backend ml_engine -ll`	Python source, medium+ severity	**No issues.** With all severities: 1 low-severity note (B105) for the documented development placeholder secret, which production start-up rejects.
`pip-audit --path <Python 3.11 environment>`	The packages installed from `requirements*.txt`	**No known vulnerabilities found** (checked 2026-11-03 against the public vulnerability database the tool queries).
`ruff check .`	Lint (pycodestyle, pyflakes, bugbear, isort, pyupgrade)	Clean
`cfn-lint`	CloudFormation templates	Clean
`npm audit`	Front-end dependencies	See `docs/15_verification_matrix.md`

A clean scan is the absence of *known* issues at one point in time, not proof of absence of vulnerabilities. The CI workflow repeats the scans on every push.

3. What was not done

- No external penetration test, no fuzzing campaign, no formal threat-modelling workshop.
- **Docker images were not built in this environment** (the container runtime starts, but the registry holding the base images is not reachable from the workspace), so the container hardening items above are reviewed, not machine-verified. The CI workflow builds and smoke-tests both images on GitHub-hosted runners.
- Nothing was deployed to AWS; the cloud controls are configuration that passed a linter.
- The login throttle, the sliding-window rate limiter and the token model are per process and stateless (no token revocation list, no refresh tokens): a stolen token works until it expires (60 min) unless the user is deactivated.
- Tokens are held in memory and `sessionStorage` by the UI, so a successful XSS could read them; mitigations are the CSP, the absence of `dangerouslySetInnerHTML`, and a short lifetime. Cookie-based sessions would remove that exposure at the cost of CSRF handling.
- TLS is terminated by the load balancer; without a certificate the AWS template serves HTTP and says so.

4. Privacy

Uploaded data is household electricity data and is personal data in many jurisdictions. It is stored per user, never shared between users, deletable by its owner (the delete endpoint removes readings, models and files), and never sent to a third party by the application. The bundled sample dataset is a one-year slice of a published research dataset (CC BY 4.0). There is no analytics or tracking code.

Chapter 15. Testing

Everything below was run for real in the build workspace (Linux, 2 CPU cores, 7 GB RAM, Python 3.11.15 with the pinned `requirements.txt`, Node 22.23, PostgreSQL 16). Numbers are copied from the command output; nothing here is estimated. What was **not** tested is listed in §7 and in `docs/15_verification_matrix.md`.

1. Test suites

Suite	Where	Tests	What it covers
Unit	`tests/unit`	84	CSV loading and ingestion (units, time conventions, bad files), quality report, feature building and leakage canaries, chronological splits, forecasting models and registry, LSTM, anomaly detector, TreeSHAP additivity, tariff cost, recommendations and scheduler, forecast residual band, secret scanner
Integration and security	`tests/integration`	46	The real FastAPI app over HTTP (TestClient) with a real database: auth, ownership (IDOR), uploads, jobs, train → forecast → explain → anomalies → optimise → monitoring, migrations (Alembic ↔ models), backup/restore, and 16 dedicated security tests (see §3)
End to end	`tests/e2e`	3	A real uvicorn process driven over real HTTP (2 tests) and a real Chromium driving the built React app against it (1 test)
Front end	`frontend/src/**/*.test.*`	122	Vitest + Testing Library against 45 fixtures captured from the running backend: pages, auth and guards, API client, polling, validation, components
Total		**255**	

Results of the final runs

Command	Result
`pytest` (all of the above Python suites, SQLite)	**131 passed, 2 skipped** in 154 s (the 2 skips need PostgreSQL)
`pytest tests/integration` with `ENERGIQ_TEST_DATABASE_URL` = PostgreSQL 16	**46 passed, 0 skipped** in 55 s (includes the PostgreSQL migration and `pg_dump`/restore tests)
`npm test` in `frontend/`	**122 passed** (8 files)
Line coverage, `pytest --cov=backend --cov=ml_engine`	**95 %** (2,971 statements, 147 missed); the e2e server runs in a subprocess and is not counted; `ml_engine/evaluation/style.py` (plot styling used by the research scripts) has 0 %

Frontend coverage was not measured. The Python suites also passed on Python 3.13 earlier in the project (126 passed, 2 skipped at that time, before the browser and reproducibility tests were added); the final run above is Python 3.11 only.

2. Properties the tests pin down (not just “it runs”)

- **No leakage.** Features use only data at or before the forecast origin; training never sees rows after the cut-off; rewriting every reading after an origin does not move the forecast (service-level and API-level canaries); splits are chronological.
- **Reproducibility (NFR-04).** Training the same dataset twice gives identical hold-out metrics, residual quantiles and forecasts, and the same data hash (`test_retraining_on_identical_data_reproduces_metrics_and_forecasts`). On a larger scale the whole research benchmark was re-run from the raw file:

baselines, Random Forest and all three LSTM seeds reproduced exactly; XGBoost depends on the library version (`docs/06_model_evaluation.md` §8).

- **Explanation correctness.** TreeSHAP contributions plus the base value equal the model output to at most 3.8×10^{-6} kW on 7,233 test rows (asserted in unit tests and measured in `shap_global.json`).
- **Forecast band.** The 10th/90th percentile band is built from `actual − predicted` residuals; four tests cover the sign convention (see bug 1 in §6).
- **Data integrity.** Append mode replaces an hour only when the new file has a value for it (`test_append_replaces_overlapping_values_but_never_erases_readings_with_gaps`; the test fails on the previous code).
- **Schema.** The Alembic migration reproduces exactly the schema of the ORM models, upgrades and downgrades on SQLite and PostgreSQL, and the production-mode app boots on a migrated database.
- **Backup and restore.** A dump/restore round trip returns identical data on SQLite and PostgreSQL; corrupted archives and path-traversal entries are refused.

3. Security tests (`tests/integration/test_security.py`, 16 tests)

Every route except health and the auth endpoints refuses an unauthenticated call; the role matrix (viewer / analyst / admin) is derived from the route table itself so a new route cannot silently skip it; security headers and CORS behaviour; mass assignment (clients cannot set `role`, `owner_id`, …); SQL-injection strings stored and returned as data; HTML report escapes user text (stored-XSS canary) under a restrictive CSP; one user cannot read, change or delete another's datasets, models, forecasts, jobs, anomalies or tariffs (IDOR) and gets 404 rather than 403; upload size, extension and content checks; login, registration and upload rate limits; a forged `X-Forwarded-For` does not evade rate limiting; error responses and job errors do not leak internals; tokens in the query string are rejected.

Static and dependency checks:

Tool	Result
`ruff check .`	clean
`bandit -r backend ml_engine -ll` (medium and high severity)	no findings; one low-severity note (B105) on the development-only placeholder secret, which production mode refuses
`pip-audit -r requirements.txt`	no known vulnerabilities (at the time of the run)
`npm audit --omit=dev --audit-level=high`	exit 0; 0 high/critical; 2 moderate (react-router 6.x, see `docs/16_frontend.md` §4)
`npm audit` (including development tools)	2 critical in `vitest/tinypool` (test runner only, never shipped; fix is a breaking major upgrade)
`cfn-lint infrastructure/aws/*.yaml`	clean
`scripts/check_secrets.py`	0 findings
`docker compose config -q`	valid

4. Measured performance and resources

Single user on `localhost`, one uvicorn process, production mode, PostgreSQL 16, 100 requests per endpoint. This is a functional measurement, **not a load test**; it supports no capacity claim.

Item	Measured
`/health/live`	p50 1.6 ms, p95 2.0 ms
`GET /datasets`	p50 5.2 ms, p95 6.3 ms
`GET /datasets/{id}/readings`	p50 8.7 ms, p95 11.4 ms
Bundled sample ingestion (8,760 hours)	0.5 s
Training the 24-model bundle on the sample	12.4 s to 25 s (varies with machine load)

Item	Measured
24-hour forecast / anomaly run on the sample	0.6 s / 0.5 s
Peak memory of the API process over the whole workflow	294 MB
Research benchmark, tree models (full 34,151 valid hours)	peak 600 MB, 743 s
Research benchmark, one LSTM seed	peak 1.1 GB, 145 s (3 seeds: 667 s)
Dataset profiling of the raw file (2,075,259 one-minute rows, 127 MB)	peak 508 MB

All of these are below the 8 GB budget (NFR-01). Peak memory of the Docker images was not measured (they were not run).

5. Browser end-to-end test

`tests/e2e/test_ui_workflow.py` registers a new user in Chromium (first with a weak password, which the form must refuse before sending anything), loads the sample, trains a model with production parameters, creates a forecast and runs anomaly detection. It asserts that the dataset hours, the 1-hour hold-out MAE and the forecast peak shown on screen equal the values the API returned, that every expected API call happened with the expected status, and that there were no 5xx responses or console errors. Screenshots go to `docs/screenshots/e2e/` (git-ignored; uploaded as CI artefacts).

A test-quality lesson. The first version of this test passed on its first run but was **vacuous**: it waited for "a table" and matched the empty *Saved forecasts* table, so it passed before any forecast existed. This was only noticed by looking at the screenshot it had saved. The test now waits for the forecast card itself and compares numbers with the API. Screenshots are therefore part of the review of any UI test.

6. Defects found during verification

1. **Forecast band sign.** The 10th/90th percentile band had been built from `predicted − actual`, which mirrored it (the wider side was below the forecast; measured coverage on a skewed synthetic case was 70.5 % against a nominal 80 %). Found while writing the documentation; fixed (`residual_band`) with four tests. Models trained before the fix keep their stored quantiles and should be retrained.
2. **Rate-limit evasion.** The client address was taken from the left-most `X-Forwarded-For` entry, which a client can forge. Fixed to count trusted hops from the right (`ENERGIQ_TRUSTED_PROXY_HOPS`), with tests.
3. **Unclosed file handles** in the CSV reader (ResourceWarning). Fixed.
4. **Append data loss.** Appending a file with gaps erased stored readings in the overlap. Fixed with a regression test that fails on the old code.
5. **Benchmark not reproducible across XGBoost builds.** Found by re-running the chain; results regenerated under the pinned environment and library versions are now recorded in every result file (`docs/06_model_evaluation.md` §8).
6. **Post-sign-in redirect** (frontend): opening a protected page while signed out landed on `/` after sign-in instead of the requested page. Found in live testing by the frontend author; fixed with two tests.
7. **Vacuous browser test** (§5).

7. Not tested

- **Docker images were never built or run** (the workspace cannot pull base images); `docker compose config` and the entrypoint script (with a stub for Alembic) were checked, nothing more. The CI job that builds them has not run.

- **No GitHub Actions run**: the repository was not pushed, so every CI job is unexecuted as a pipeline (each step was run locally instead, as listed above).
- **No AWS deployment**; the CloudFormation templates only passed `cfn-lint`.
- **Windows** was not used; **browsers other than Chromium**, screen readers, slow networks and phones (beyond emulated viewport sizes) were not tested.
- **No load or soak testing**; no multi-process deployment testing (the rate limiter and job runner are single-process).
- **Anomaly accuracy on real faults** cannot be tested: the dataset has no labels (synthetic injection only).
- **Frontend coverage** was not measured.

Chapter 16. Deployment and operations

1. Status ladder (what has really been done)

Level	Meaning	Status in this project
1. Implemented locally	Runs and is tested on a development machine	**Yes**: API, ML engine and (see the verification matrix) the web UI run and are tested, on Python 3.11 with SQLite and with PostgreSQL 16
2. Docker-tested	Images built and run as containers	**No.** Dockerfiles and `docker-compose.yml` are written and `docker compose config` accepts the compose file, but the images were **not built or run**: the container runtime starts in the build workspace but cannot pull base images (registry blocked by the workspace's network policy). The CI workflow builds both images and smoke-tests the API image on GitHub-hosted runners; that job has not run either, because the repository has not been pushed.
3. Cloud-configured	Infrastructure-as-code and pipeline files exist and pass static checks	**Yes**: `infrastructure/aws/ecr.yaml` and `energiq-stack.yaml` pass `cfn-lint`; `.github/workflows/ci.yml` and `deploy-aws.yml` parse as valid YAML
4. Actually deployed	A live environment exists	**No.** No AWS account or resources were used.
5. Independently verified	Someone other than the author tested it	**No.**

Nothing in this repository should be described as "deployed on AWS" or "Docker-tested". Any claim must stay at the level reached above.

2. Local development (Windows, macOS, Linux)

Requirements: Python 3.11, Node.js 22, 8 GB RAM, no GPU.

```
python -m venv .venv
.venv\Scripts\activate                          (Linux/macOS:
source .venv/bin/activate)
pip install -r requirements-dev.txt
set ENERGIQ_ENV=dev                             (Linux/macOS: export ...)
set ENERGIQ_ADMIN_EMAIL=admin@example.com
set ENERGIQ_ADMIN_PASSWORD=choose-a-long-password-1
uvicorn backend.app.main:create_app --factory --reload --port 8000
```

```
cd frontend
npm ci
npm run dev                                   (http://localhost:5173,
proxies /api to port 8000)
```

Development uses SQLite (`data/energiq.db`, created automatically). The LSTM benchmark scripts additionally need `pip install -r requirements-research.txt` (PyTorch CPU).

3. Local production-like stack (Docker Compose)

```
copy .env.example .env                        (fill POSTGRES_PASSWORD and
ENERGIQ_SECRET_KEY)
docker compose up --build
```

Serves the UI on `http://localhost:8080`; nginx proxies `/api` to the API container, which waits for PostgreSQL, applies Alembic migrations (`docker/backend-entrypoint.sh`) and starts one Uvicorn worker. Data lives in the named volumes `pgdata` and `appdata`. *This procedure is documented but was not executed here (see §1).*

4. Database migrations

```
alembic -c backend/alembic.ini upgrade head          # uses
ENERGIQ_DATABASE_URL
alembic -c backend/alembic.ini revision --autogenerate -m "describe
change"
```

`dev` and `test` environments create tables automatically; **`prod` never does**, it expects `upgrade head`. A test checks that the migration builds exactly the schema the models describe and that it can be downgraded (SQLite and PostgreSQL 16).

5. AWS (configuration only)

Two templates, deployed in order, with the AWS CLI or the manual GitHub workflow (`deploy-aws.yml`, `workflow_dispatch` only).

1. `ecr.yaml`: two image repositories (immutable tags, scan on push, lifecycle policy, retained on stack deletion).
2. `energiq-stack.yaml`: VPC with two public and two private subnets (no NAT gateway); ALB (HTTPS only if you pass an ACM certificate ARN, otherwise HTTP with a stated warning); ECS Fargate services (API ×1, web ×2); RDS PostgreSQL 16 (encrypted, 7-day backups, not publicly accessible, password generated and held by RDS in Secrets Manager); EFS (encrypted) for uploads and models; generated JWT secret in Secrets Manager; log groups; alarms (API 5xx, no healthy API task, database CPU and free storage) to an SNS topic.

Before any real use:

- Confirm the PostgreSQL minor version (`aws rds describe-db-engine-versions --engine postgres`) and adjust `EngineVersion`.
- Supply a certificate; restrict `AllowedIngressCidr`; consider `DbMultiAz=true`.
- The stack creates billable resources (ALB, Fargate tasks, RDS, EFS, log storage). Estimate with the AWS Pricing Calculator first; delete the stack when finished (the database takes a final snapshot and the EFS file system and repositories are retained and must be deleted by hand).
- The API service is intentionally limited to one task (see `docs/13_limitations.md`).

No stack was created; no cost was incurred.

6. Configuration reference

All variables use the `ENERGIQ_` prefix.

Variable	Default	Notes
`ENV`	`dev`	`dev`, `test` or `prod`. `prod` disables `/docs`, enables HSTS and refuses unsafe settings
`DATABASE_URL`	`sqlite:///./data/energiq.db`	`postgresql+psycopg://user:pw@host:5432/energiq` in production (the image can also assemble it from `DB_HOST, DB_USER, DB_PASSWORD, DB_NAME, DB_PORT`)
`SECRET_KEY`	development placeholder	≥ 32 random characters in `prod`
`ACCESS_TOKEN_MINUTES`	60	JWT lifetime
`CORS_ORIGINS`	localhost dev origins	JSON list; `*` refused in `prod`
`ALLOW_REGISTRATION`	true	Set false to create users only through the administrator
`ADMIN_EMAIL`, `ADMIN_PASSWORD`	unset	If both set, an administrator is created on first start; change the password afterwards
`UPLOAD_DIR`, `MODELS_DIR`	`data/uploads`, `artifacts/models`	Must be on persistent storage
`MAX_UPLOAD_MB`	50	The nginx limit is 210 MB
`TRUST_PROXY`, `TRUSTED_PROXY_HOPS`	false, 1	Enable only behind proxies you control
`JOB_WORKERS`	2	Background threads
`METRICS_ENABLED`	true	Serves `/metrics`; do not expose it publicly (neither nginx nor the ALB rules route it)

Chapter 17. MLOps and monitoring

1. What is tracked

Item	Where	Notes
Model versions	`model_versions` table (+ files under the models directory)	Dataset, horizon-model ids, train end, hash of the training series, hold-out metrics against baselines, residual quantiles, SHAP summary, active flag
Reproducibility	`artifacts/reports/benchmark/cv_results.json`	Seed, folds, data hash and run time of the research benchmark
Forecasts	`forecasts` table	Every forecast is stored with its origin and model, which enables realised-error tracking
Audit trail	`audit_events`	Who did what, from which IP
Notifications	`notifications`	Job completions and anomaly runs

There is no separate experiment-tracking server (MLflow was deliberately left out to keep the stack light for an 8 GB machine); metadata lives in the application database and the JSON reports.

2. Model monitoring (`GET /api/v1/models/{id}/monitoring`)

- **Data drift.** Compares the load distribution of the training period with the most recent 28 days using the Population Stability Index (reference-quantile bins) and the two-

sample Kolmogorov-Smirnov test. Status: PSI < 0.1 “stable”, 0.1–0.25 “moderate shift”, ≥ 0.25 “significant shift”. These are the customary rules of thumb, not validated cut-offs for this domain. Drift is only reported as meaningful when new data has arrived after training; otherwise the recent window is just the tail of the training data and the response says so.

- **Realised performance.** For the latest 50 stored forecasts, the absolute error against actual readings that have since arrived. The ratio to the mean hold-out MAE (over h = 1, 6, 24) is classified “within expectation” (≤ 1.25) or “degraded – consider retraining”. The response notes that the ratio mixes horizons and is a rough alarm.
- **Retrain recommendation.** `retrain_recommended` is true when drift is significant or realised error is degraded. Retraining is manual (the user starts a training job, then activates the new version); old versions are kept until deleted, so rollback is “activate the previous version”.

3. Service monitoring

- `/health/live` and `/health/ready` for orchestrators; the Docker image and the ALB use `/health/ready`.
- `/metrics` in Prometheus text format: `energiq_http_requests_total{method,route,status}`, `energiq_http_request_duration_seconds` histogram, `energiq_jobs{status}`. Route labels are route *templates* (`/api/v1/datasets/{dataset_id}`), so cardinality stays bounded. Keep the endpoint on the private network.
- `GET /api/v1/admin/system` shows counts, job states and request/latency summary (admin only).
- Logs are plain text on stdout via Python’s `logging` and Uvicorn (collected by Docker or CloudWatch); there is no JSON log formatter; each response carries `X-Request-ID` and unhandled errors log that id.
- CloudWatch alarms in the AWS template: API 5xx, no healthy API task, database CPU, database free storage.
- Suggested Prometheus alert rules (not shipped as running configuration): 5xx rate > 1 % for 5 min; p95 latency > 2 s for 10 min; `energiq_jobs{status="failed"}` increasing.

4. Backup and restore

`scripts/backup.py` creates one `tar.gz` with the database (SQLite online backup or `pg_dump -Fc`), the uploads and models directories, and a checksum manifest. `restore` verifies the checksums, refuses unsafe archive members and different database backends, and asks for confirmation. Tests cover the SQLite and PostgreSQL round trips, corruption detection and path-traversal refusal. Recommended practice (not automated here): run the backup daily from a scheduler, copy archives off the machine, and rehearse a restore. The AWS template additionally enables 7-day automated RDS backups and EFS backups.

5. Release and change process

1. Branch, change, run `ruff check .` and the full test suite locally.
2. CI (`.github/workflows/ci.yml`) repeats lint, unit and integration tests with a coverage report (SQLite, then PostgreSQL), security scans, front-end checks, an end-to-end job (real server plus real Chromium), image builds and a CloudFormation lint. None of these CI jobs has run on GitHub yet because the repository has not been pushed; every check was run locally instead (`docs/11_testing.md`).

3. Database changes go through an Alembic revision, reviewed with the code.
4. Deployment is manual (`deploy-aws.yml`), behind a protected GitHub environment.

Chapter 18. Limitations and future scope

1. Limitations (what a reader should not over-trust)

Data and modelling

1. **One household, one source.** All benchmark results come from a single French household's 4-year record. They say nothing about other buildings, and the application's per-dataset training is evaluated only on that dataset's own final block.
2. **Modest accuracy at longer horizons.** MAE is 32–42 % of mean load and R^2 is about 0.40 at 6 and 24 h (`docs/06_model_evaluation.md`). The models beat naive baselines by roughly 18–25 % MAE; they are not precise predictors of individual hours.
3. **Deep learning showed no benefit.** The LSTM equals XGBoost at 6 and 24 h and is worse at 1 h. A larger architecture search, more input channels (sub-metering, weather) or more data could change this; it was not explored.
4. **No exogenous drivers.** No weather, occupancy or public-holiday data is used. Holidays and heat waves are therefore out-of-pattern events for the models.
5. **Anomaly detection has no ground truth.** Only the chosen alert rate (~2 % of hours) and a synthetic injection check exist; moderate drops are often missed (`docs/08_anomaly_detection.md`). Detected events are leads, not diagnoses.
6. **Forecast range is empirical.** The 80 % band comes from hold-out residual quantiles and was not validated for coverage on independent data.
7. **Recommendations are heuristic scenarios.** Savings figures assume fixed percentages and a user-supplied tariff; the built-in DEMO tariff is illustrative only.
8. **An atypical period was kept without a sensitivity check.** The August 2008 ten-day low-use block (`docs/04_eda_report.md`) stayed in the training data; the planned with/without comparison was not run, so its influence on the models is unknown.
9. **Time conventions.** Timestamps are treated as the dataset's own local clock; daylight-saving effects are only checked for the benchmark dataset.

Software and operations

10. **Single API process.** The rate limiter, the job runner's thread pool and the metrics registry are in memory. Running several API workers or tasks would split rate-limit counters, lose job progress visibility across processes, and give per-process metrics. The deployment is therefore limited to one API instance (the compose file and ECS service are set accordingly). This caps availability (a restart interrupts running jobs) and throughput.
11. **Training is CPU-bound and synchronous in a thread.** Measured on the bundled one-year sample (8,760 hourly rows) on a 2-core, 7 GB machine: training the 24-model bundle took about 25 s, a 24-hour forecast 0.6 s and anomaly detection 0.5 s. Several years of data or a slower machine will take proportionally longer, and training competes with API requests for CPU.
12. **Token model.** Stateless access tokens without refresh or revocation lists; the UI keeps the token in memory/`sessionStorage`.
13. **File storage is local.** Uploads and models are on a mounted volume (EFS in the AWS template), not an object store; there is no virus scanning of uploads (they are only parsed as text numbers).

14. **Not validated at scale.** Tests use datasets of up to a few years of hourly data; there was no load testing beyond functional tests, so no capacity or latency guarantees are made.
15. **Not run in containers or in the cloud here** (`docs/12_deployment.md` §1).
16. **Python versions.** The suite was run on Python 3.11 (supported) and 3.13; other versions are untested.
17. **Browsers.** The UI was exercised in Chromium only.
18. **Results depend on library versions.** Baselines, Random Forest and the LSTM reproduced exactly when the whole benchmark was re-run, but XGBoost's predictions moved by up to 0.0036 kW test MAE between XGBoost builds (3.2.0 pinned here, 3.4.1 in the system Python 3.13). A different build can also change which CV configuration is selected and flip the sign of very small model differences (`docs/06_model_evaluation.md` §8). Results are bit-reproducible only with the pinned `requirements.txt`; every result file records its versions.
19. **Windows is untested.** The commands in the manuals are written to work on Windows (plain npm scripts, no shell-only syntax in Python scripts, path handling via `pathlib`), but nothing was run on Windows; all verification was done on Linux.

2. Future scope

- Add weather and calendar (holiday) features and quantify their effect with the same chronological protocol; evaluate on more households (for example other public smart-meter data sets) to test generality.
- Probabilistic forecasting (quantile regression or conformal intervals) with measured coverage, replacing the empirical band.
- Label collection: let users confirm or dismiss anomalies, then evaluate and tune the detector against those labels and consider a supervised or semi-supervised model.
- Appliance-level insights from sub-metering where the data has it.
- Externalise state: Redis (or the database) for rate limiting, a real queue (Celery, RQ or SQS) for jobs, so the API can scale horizontally; object storage for files.
- Scheduled retraining with a champion/challenger comparison and automatic rollback, instead of the current manual retrain recommendation.
- Real tariff integration (time-of-use and dynamic prices), load-shifting optimisation across several devices, and battery/PV scenarios.
- Cookie-based sessions with CSRF protection, refresh tokens, multi-factor authentication, and an independent security review.
- Accessibility audit with assistive technology, internationalisation of the UI.

Chapter 19. Verification matrix

Status values: **PASS** (checked by an automated test or a measurement that was run, evidence named), **PARTIAL** (works, with a stated scope limit), **NOT TESTED** (written but never run or exercised), **NOT TESTABLE** (cannot be measured with the data available). Nothing is marked PASS on the basis of "it should work". Test counts and commands are in `docs/11_testing.md`. Everything was verified on Linux in the build workspace.

1. Deployment level (the ladder from `docs/03_architecture.md` §8)

Level	Status	Evidence / reason
Implemented locally	**YES**	Runs on Linux with SQLite and PostgreSQL 16, real HTTP, real browser (`tests/e2e`).
Docker-tested	**NO**	Dockerfiles and compose file are written;

Level	Status	Evidence / reason
		`docker compose config -q` validates. The images were never built or run (base images could not be pulled in the build workspace).
Cloud-configured	**YES**	CloudFormation templates and a manual-only deploy workflow exist and pass `cfn-lint`.
Actually deployed	**NO**	Nothing was deployed; no AWS resources were created.
Independently verified	**NO**	No one other than the author and an AI assistant has run the project; CI has never executed on GitHub.

2. Functional requirements

ID	Requirement	Status	Evidence
FR-01	Register and log in	**PASS**	`test_api_core.py`, `test_security.py` (rate limits, hashing), HTTP e2e, browser e2e (weak-password refusal and registration), frontend auth tests (18)
FR-02	Upload CSV datasets	**PASS**	16 ingest + 14 loading unit tests, upload API tests, UI upload verified live by the frontend author (upload, append, delete)
FR-03	Validate and explore data (quality report)	**PASS**	Quality report unit tests; API returns report; dataset page shows it (browser e2e checks the dataset summary against the API)
FR-04	View historical consumption patterns	**PASS**	Readings (hourly/daily) and profile endpoints tested; explorer and table view verified live by the frontend author; not covered by the repository's browser e2e
FR-05	Forecast 1 h, 6 h, 24 h (and 1–24 h curve)	**PASS**	Benchmark at 1/6/24 h (`docs/06`); API forecast tests incl. leakage canary; browser e2e creates a forecast and checks its peak
FR-06	Compare forecasting models	**PARTIAL**	The research benchmark compares 3 baselines, Random Forest, XGBoost and LSTM with bootstrap intervals and is shown on the Benchmark page. Inside the application each trained model is compared with the baselines only, not with Random Forest or LSTM
FR-07	Detect abnormal consumption	**PASS** (function) / **NOT TESTABLE** (accuracy)	10 unit tests, API flow, synthetic injection check (`docs/08` §5); the real dataset has no labels, so no accuracy is claimed
FR-08	Explain predictions	**PASS**	TreeSHAP additivity ≤ 3.8×10^{-6} kW on 7,233 rows; local and global API tests; explainability page verified live
FR-09	Cost with editable tariffs	**PASS**	Tariff/cost unit and API tests (validation, DEMO labelling, ownership); tariff

ID	Requirement	Status	Evidence
			create/delete verified live. Costs are only as good as the tariff the user enters
FR-10	Transparent optimisation, separate from anomalies	**PASS**	Rule-based recommendations carry evidence and assumptions (unit + API tests); scheduler is an exhaustive search (unit tests)
FR-11	Alerts and historical analysis	**PASS**	Anomaly events with acknowledge, notifications, jobs, audit log (API tests; UI verified live)
FR-12	Export reports	**PASS**	HTML report (escaped, CSP; test_security) and CSV export (API tests); authenticated downloads verified live
FR-13	Manage datasets and experiments	**PASS**	Create/append/activate/ delete datasets and models; deletion cascades (test_delete_dataset_removes_ everything)
FR-14	Monitor model performance	**PASS**	Drift (PSI/KS) and realised-error monitoring (API tests); monitoring card verified live. No scheduled retraining
FR-15	Admin panel and settings	**PARTIAL**	Admin users/roles, audit log and system stats exist and are role-protected (tests, verified live). Per-user settings are limited to theme, active dataset and tariffs; there is no general settings page

3. Non-functional requirements

ID	Requirement	Status	Evidence
NFR-01	8 GB RAM, CPU only, Windows	**PASS** (memory, CPU) / **NOT TESTED** (Windows)	Peak 294 MB (API workflow), 600 MB (tree benchmark), 1.1 GB (LSTM seed), 508 MB (profiling); no GPU used. Never run on Windows
NFR-02	Chunked loading, no needless full loads	**PASS**	Streaming CSV reader; profiling a 2.08-million-row file peaked at 508 MB
NFR-03	Small footprint (SQLite, one process, no Redis/Celery)	**PASS** (local) / **NOT TESTED** (compose)	Default run is one process on SQLite; compose file validated but not run
NFR-04	Reproducible training	**PASS** (with pinned libraries)	Repeat-training test; benchmark re-run reproduces baselines, RF and LSTM exactly; XGBoost differs across XGBoost versions, hence the pins and recorded versions (`docs/06` §8)
NFR-05	No leakage, chronological splits only	**PASS**	Feature/split unit tests, API canary that rewrites the future, test period scored once
NFR-06	Hashing, authn, authz, isolation	**PASS**	Argon2id; 16 security tests incl. role matrix over every route and IDOR checks
NFR-07	Upload limits, safe handling, CSV validation	**PASS**	Size/extension/content tests; client file name never used on disk; rate limit on uploads
NFR-08	No secrets in code, logs or Git	**PASS** (local)	Secret scan 0 findings, tests for the scanner, production refuses the dev secret. The CI

ID	Requirement	Status	Evidence
			job that repeats it has not run
NFR-09	Demo data labelled; no hard-coded statistics in the UI	**PASS** (by lint rule, review and tests)	DEMO tariff labelled in API and UI; lint forbids `Math.random`; UI tests run against captured API responses. There is no automated scan for numeric literals in components
NFR-10	Honest deployment status	**PASS**	§1 above and `docs/12_deployment.md`

4. Phases of the specification

Phase	Deliverable	Status	Where
1	Requirements, dataset suitability, architecture, repository	Done	`docs/01–03`, repository layout
2	Dataset profiling and EDA	Done	`docs/04`, `artifacts/reports/eda`, `dataset_profile.json`
3	Preprocessing and feature engineering	Done, tested	`ml_engine/data`, `ml_engine/features`; unit tests
4	Baseline forecasting	Done, tested	`ml_engine/forecasting`; `docs/05–06`
5	ML models (Random Forest, XGBoost)	Done, tested	CV protocol in `docs/05`; results `docs/06`
6	Lightweight LSTM	Done, tested	`ml_engine/forecasting/lstm.py`; 3 seeds, exact reproduction
7	Model evaluation	Done	`docs/06`, figures; regenerated under pinned environment
8	Anomaly detection	Done (no labels)	`docs/08`
9	Explainability and optimisation	Done, tested	`docs/07`
10	Backend and database	Done, tested (SQLite and PostgreSQL)	`docs/09`, Alembic migration
11	Frontend	Done, tested (Chromium)	`docs/16`
12	Integration and security	Done, tested	`docs/10`, security tests; defects found and fixed (`docs/11` §6)
13	Testing and cloud deployment	Testing done; **cloud deployment not done** by design (no paid resources without approval)	`docs/11`, `docs/12`, `infrastructure/`
14	Monitoring and academic deliverables	Done (monitoring implemented; no external monitoring stack deployed)	`docs/14`, `docs/academic/`, report and slides

5. Academic deliverables (spec §12)

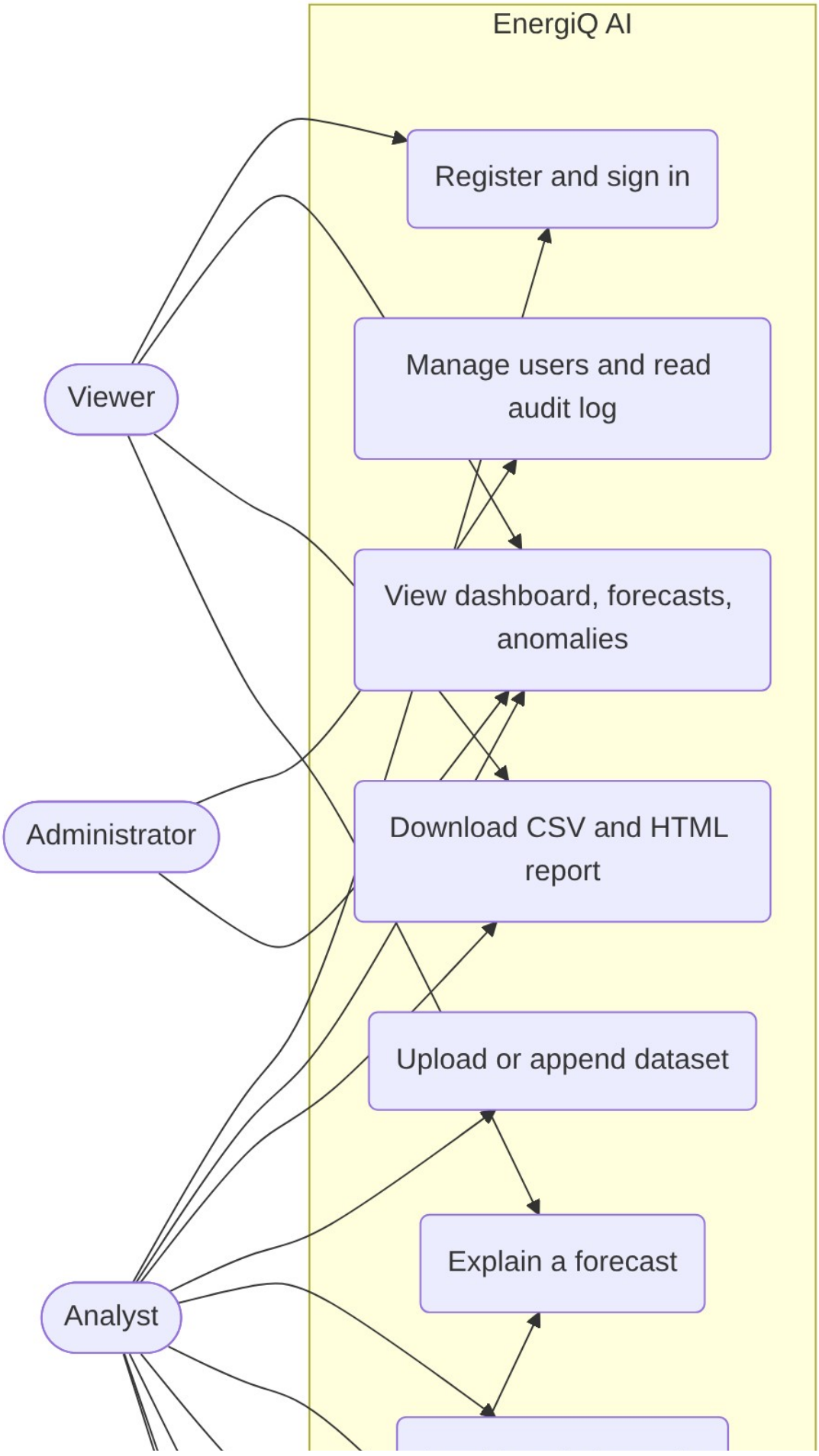
Deliverable	Where
Abstract, problem statement, objectives	`docs/academic/01–03`
Literature review (17 verified references only)	`docs/academic/04_literature_review.md`, `docs/references.md`
System requirements	`docs/02_requirements.md`
Architecture, ER, UML, data-flow diagrams	`docs/03_architecture.md`, `docs/diagrams/` (ER diagram generated from the ORM)

Deliverable	Where
ML methodology, comparison report, results	`docs/05`, `docs/06`, `docs/07`, `docs/08`
Security documentation	`docs/10_security.md`
API documentation	`docs/api/API_REFERENCE.md` (generated from the live OpenAPI, 47 operations), `openapi.json`
Installation, user manual, deployment guide	`docs/manuals/installation.md`, `docs/manuals/user_guide.md`, `docs/12_deployment.md`
Limitations and future scope	`docs/13_limitations.md`
Final report	`docs/academic/EnergiQ_AI_Final_Report.docx` (assembled from the chapters by `scripts/build_report.py`)
Presentation slides	Published as a Slides artifact (see the delivery message)
Live demonstration script	`docs/manuals/demo_script.md`

6. Things only you can do

1. Push the repository to GitHub and look at the first CI run (it has never run).
2. Build and run the Docker images (`docker compose up --build`) and then update `docs/12_deployment.md`.
3. Run the installation steps once on Windows and note any difference.
4. Replace the DEMO tariff with a real one before drawing cost conclusions.
5. Decide whether to approve any AWS deployment (it creates paid resources); nothing was created.
6. Have someone else review the literature review and the claims in the final report.

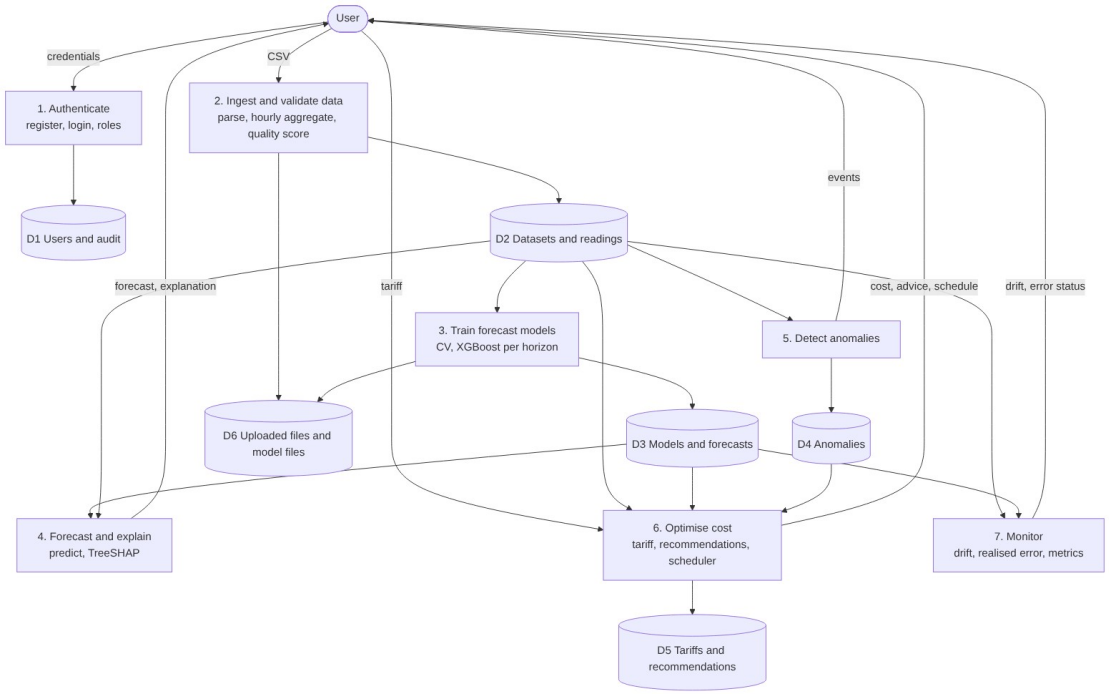
Appendix A. Design diagrams


EnergiQ AI
Register and sign in
Manage users and read audit log
Viewer
View dashboard, forecasts, anomalies
Administrator
Download CSV and HTML report
Upload or append dataset
Explain a forecast
Analyst

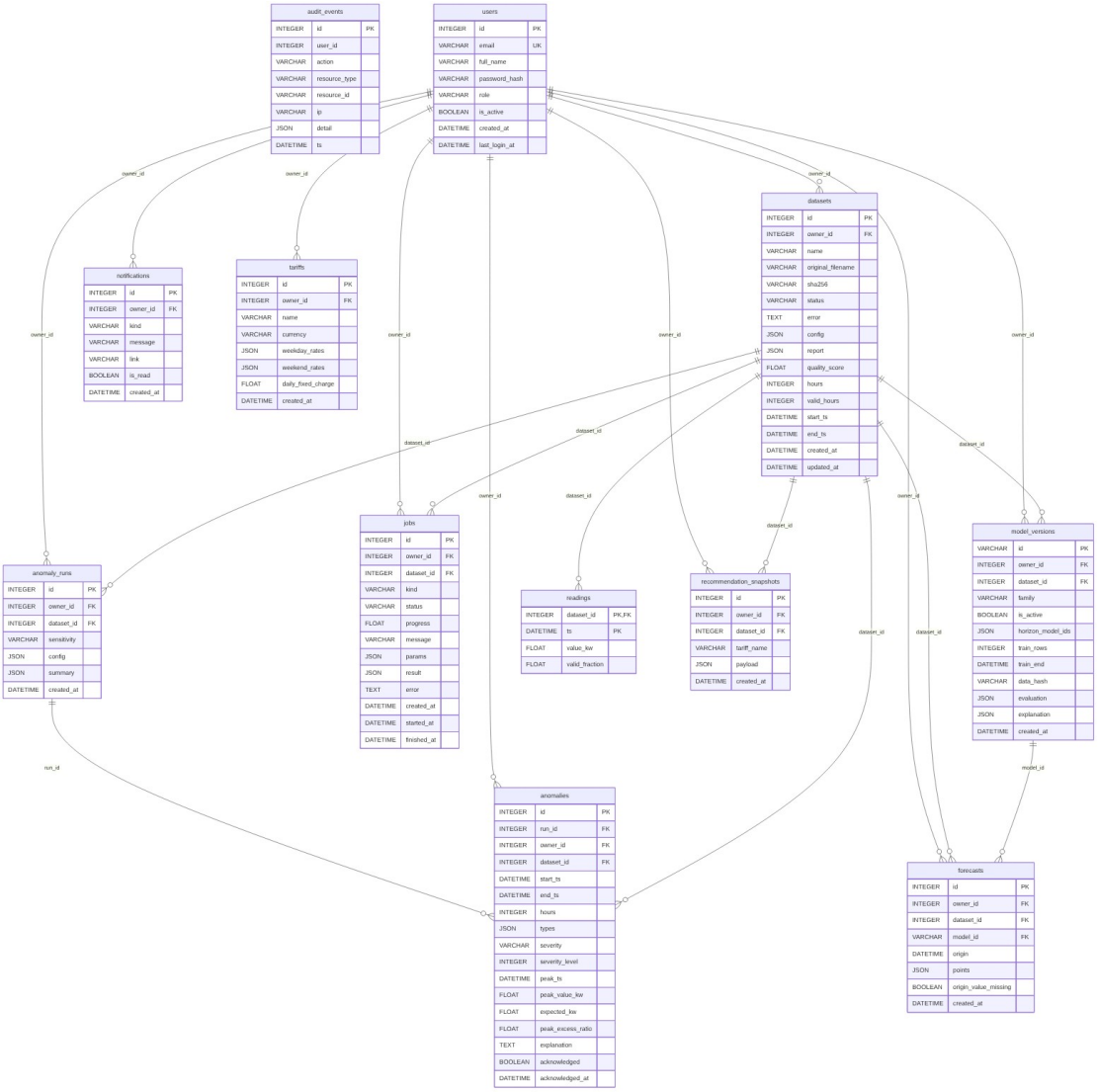
Use cases



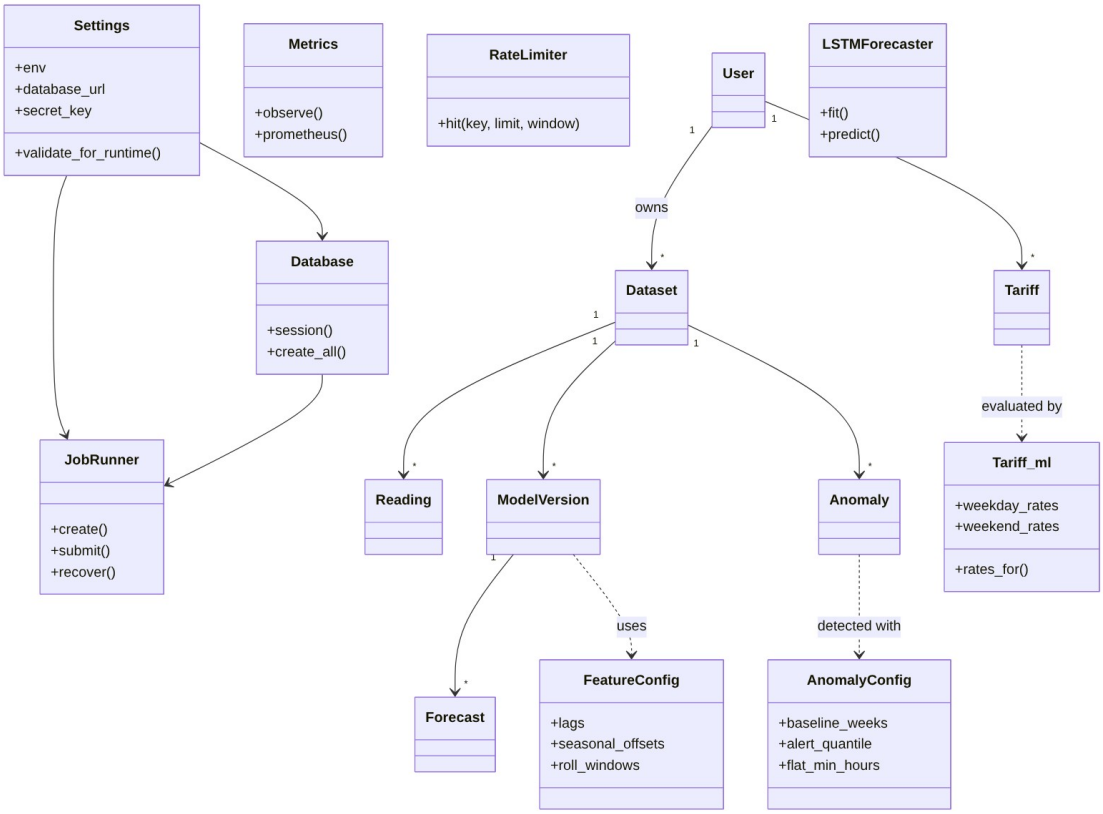
Data flow, level 0



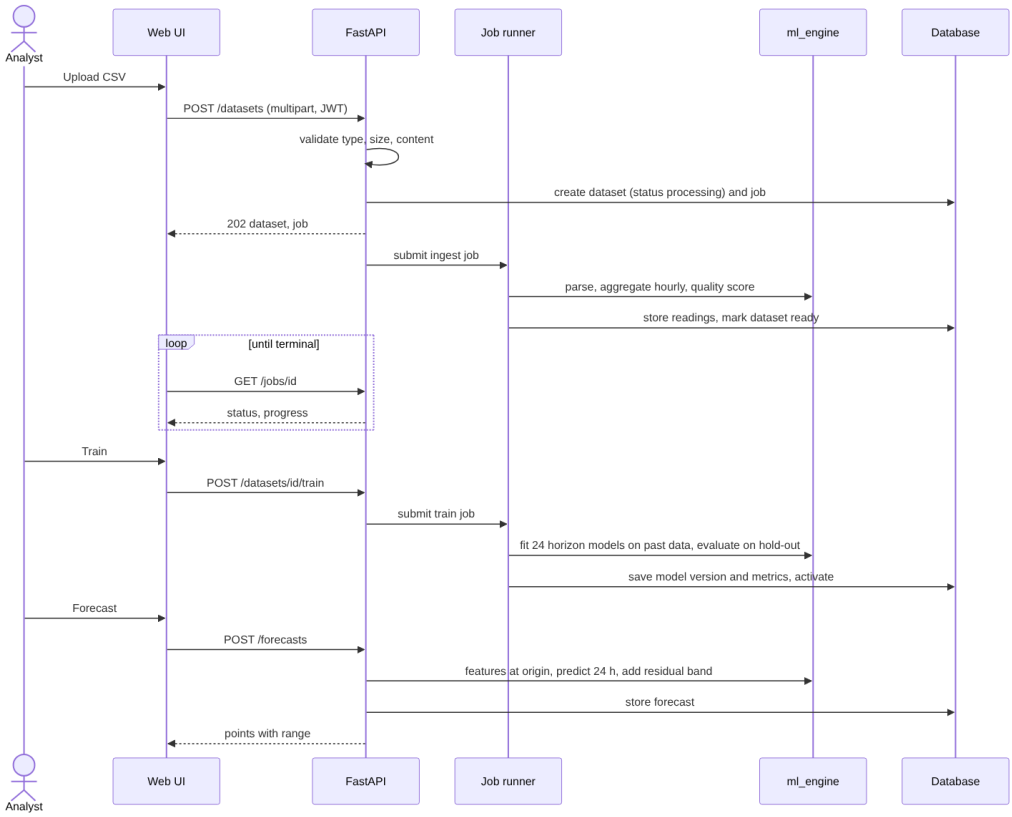
Data flow, level 1



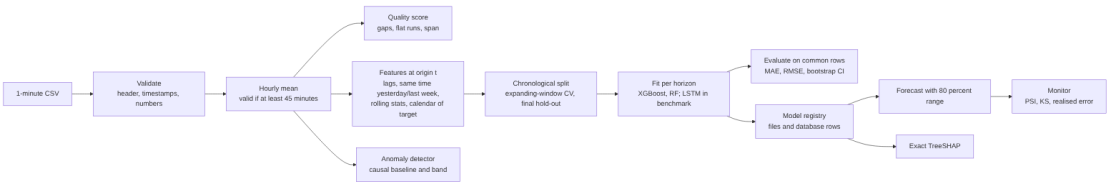
Entity-relationship diagram (generated from the ORM)



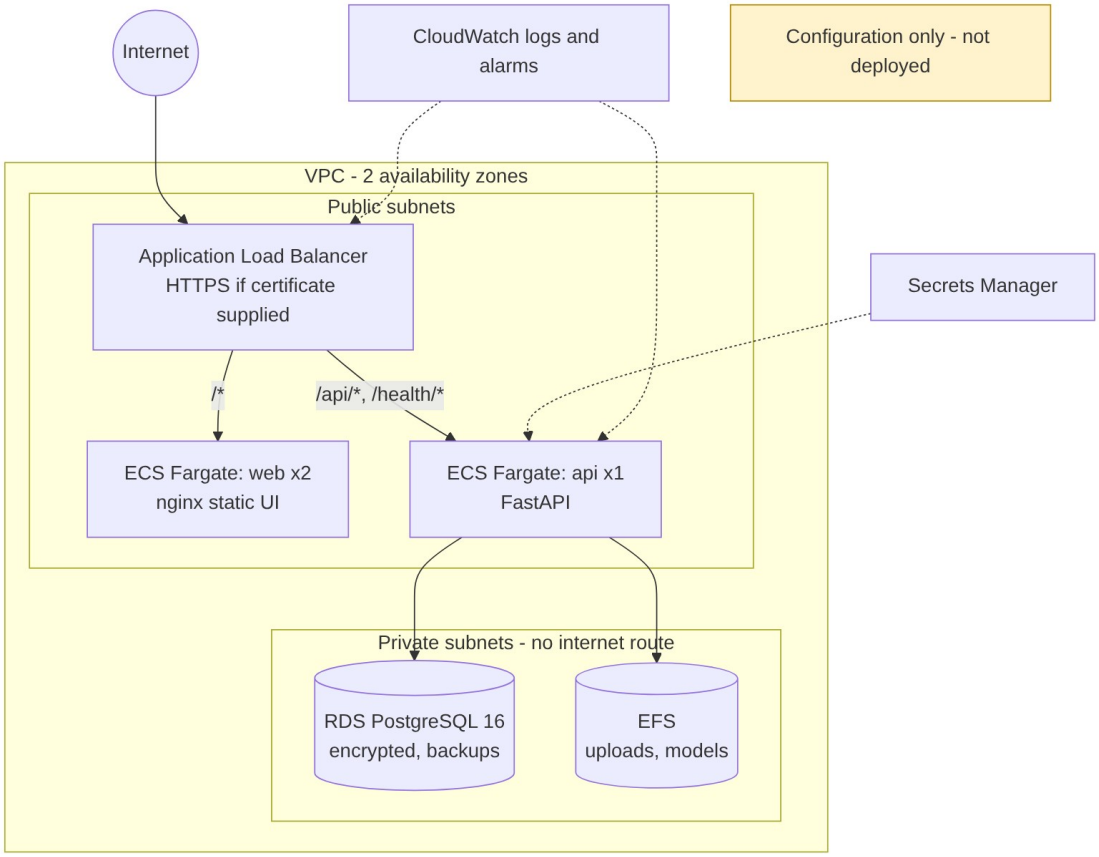
Class diagram (main classes)



Sequence: upload, train, forecast



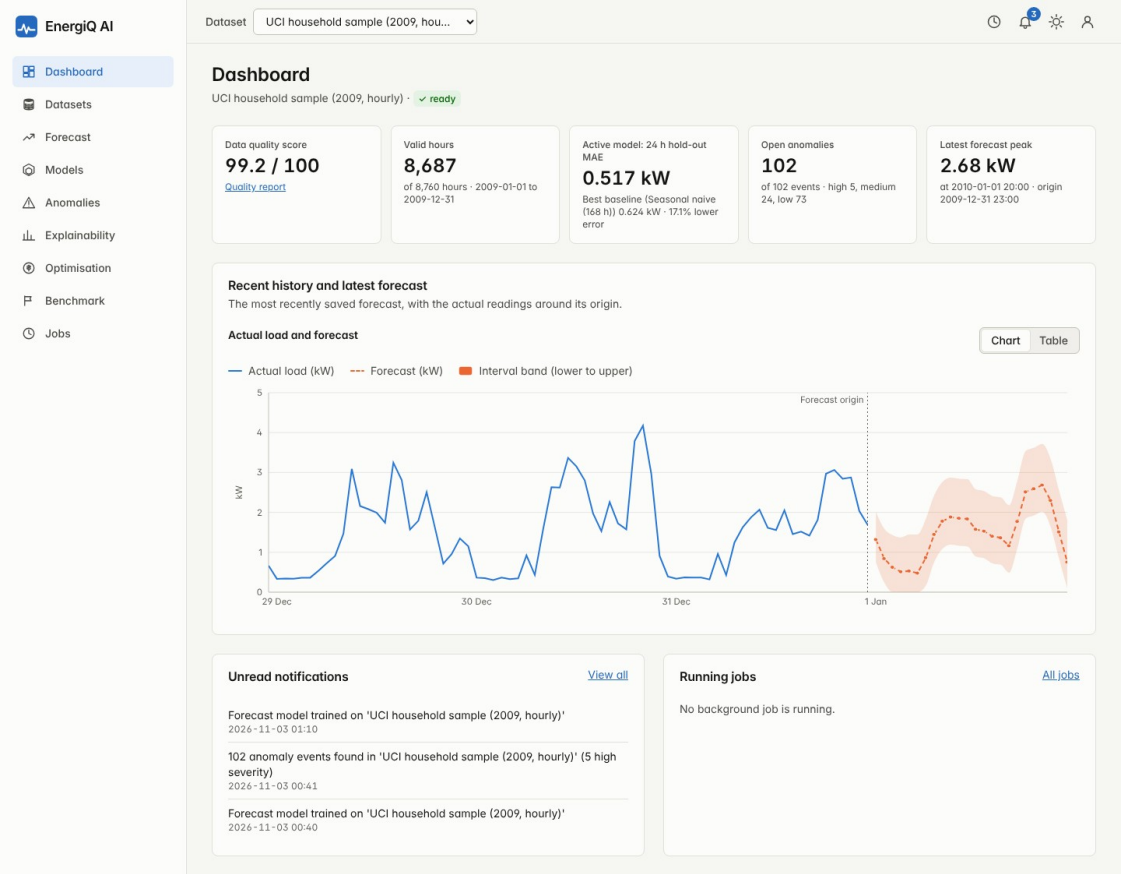
Machine-learning pipeline



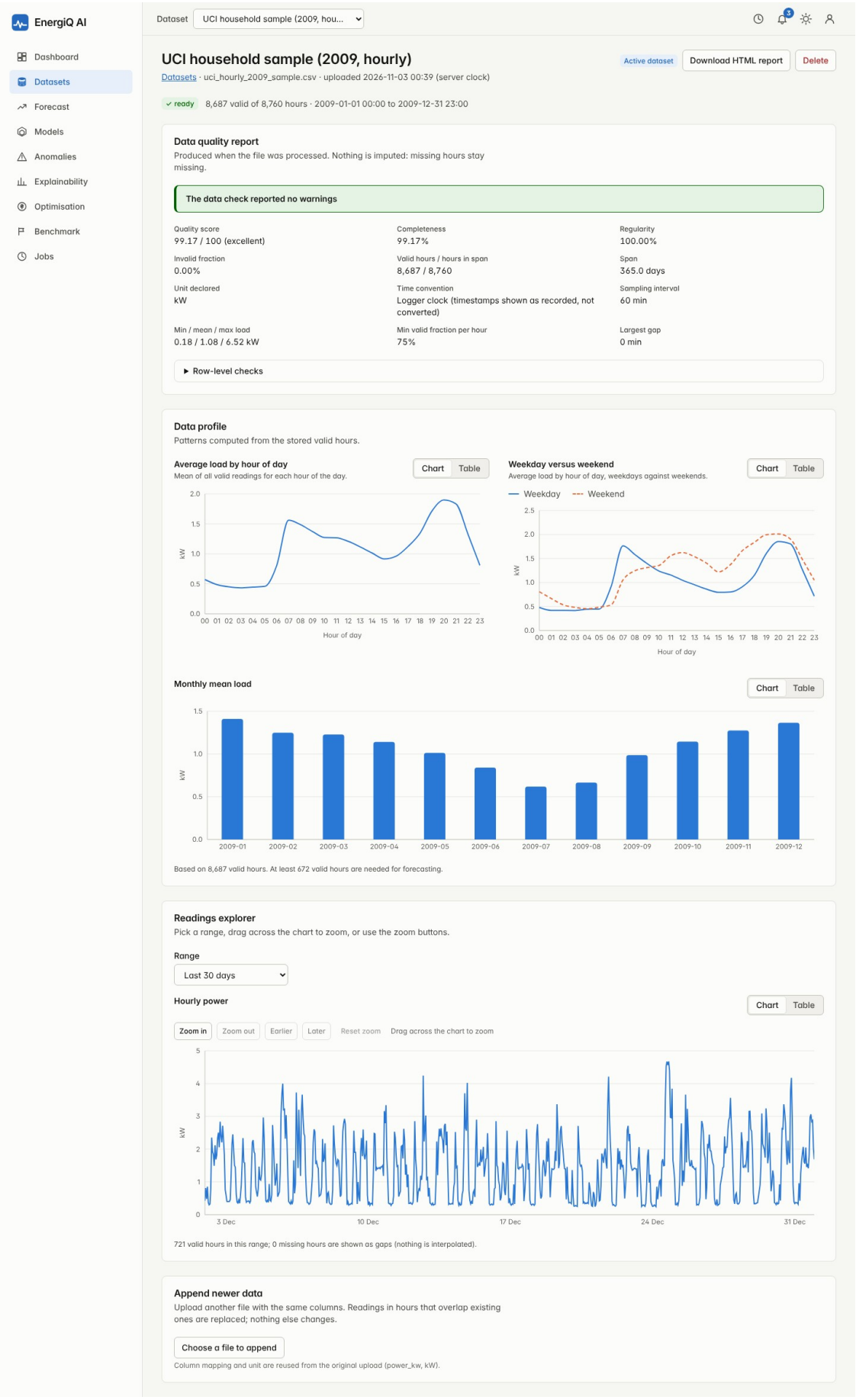
AWS deployment (configuration only, not deployed)

Appendix B. Screenshots of the running application

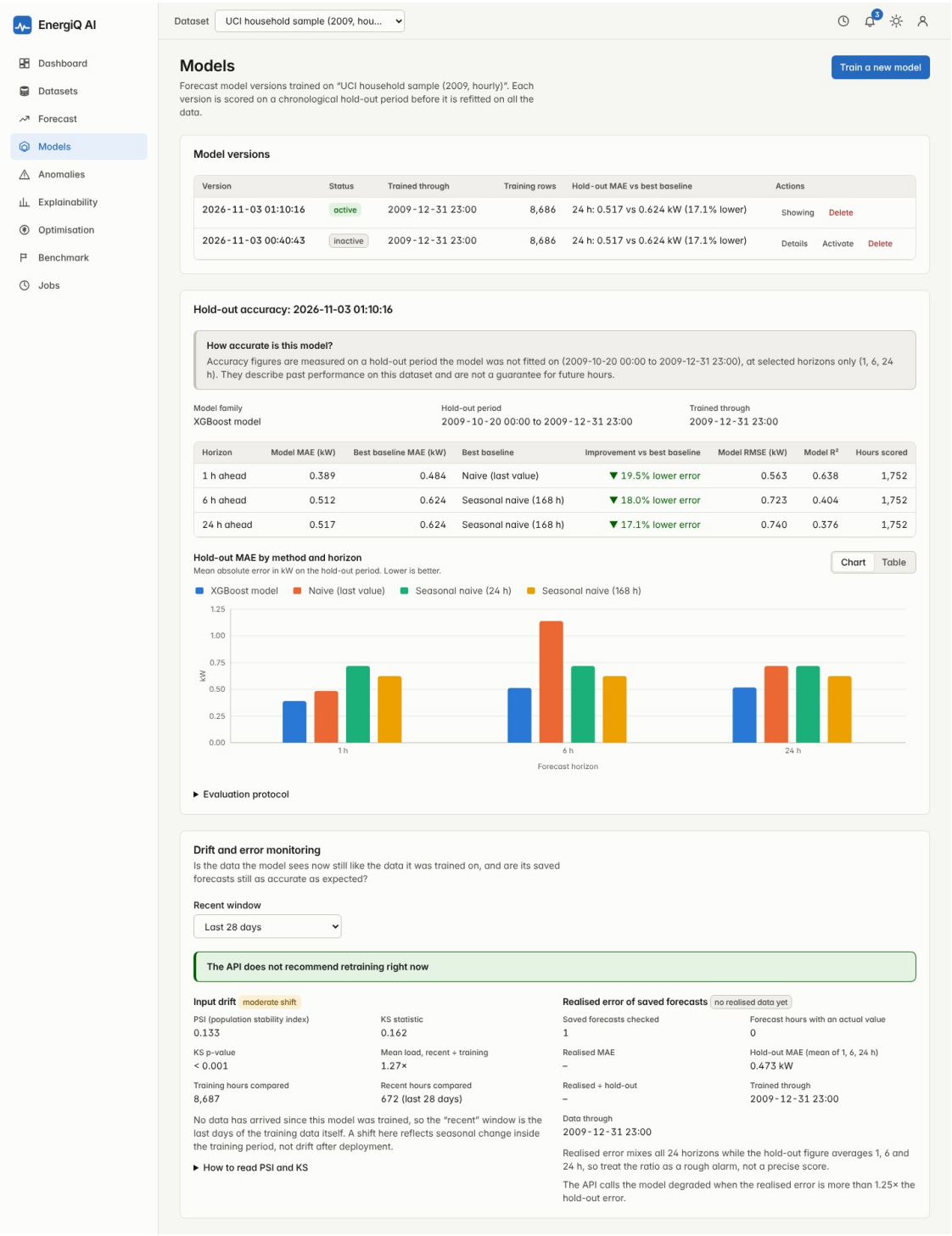
Taken in Chromium against a real backend with the bundled sample dataset (`docs/screenshots/`).


EnergiQ AI
Dataset
UCI household sample (2009, hou...
Dashboard
Datasets
Forecast
Models
Anomalies
Explainability
Optimisation
Benchmark
Jobs
Dashboard
UCI household sample (2009, hourly) · ready
Data quality score
99.2 / 100
Quality report
Valid hours
8,687
of 8,760 hours · 2009-01-01 to 2009-12-31
Active model: 24 h hold-out MAE
0.517 kW
Best baseline (Seasonal naive (168 h)) 0.624 kW · 17.1% lower error
Open anomalies
102
of 102 events · high 5, medium 24, low 73
Latest forecast peak
2.68 kW
at 2010-01-01 20:00 · origin 2009-12-31 23:00
Recent history and latest forecast
The most recently saved forecast, with the actual readings around its origin.
Actual load and forecast
Chart
Table
Actual load (kW)
Forecast (kW)
Interval band (lower to upper)
Forecast origin
kW
29 Dec
30 Dec
31 Dec
1 Jan
Unread notifications
View all
Forecast model trained on 'UCI household sample (2009, hourly)'
2026-11-03 01:10
102 anomaly events found in 'UCI household sample (2009, hourly)' (5 high severity)
2026-11-03 00:41
Forecast model trained on 'UCI household sample (2009, hourly)'
2026-11-03 00:40
Running jobs
All jobs
No background job is running.

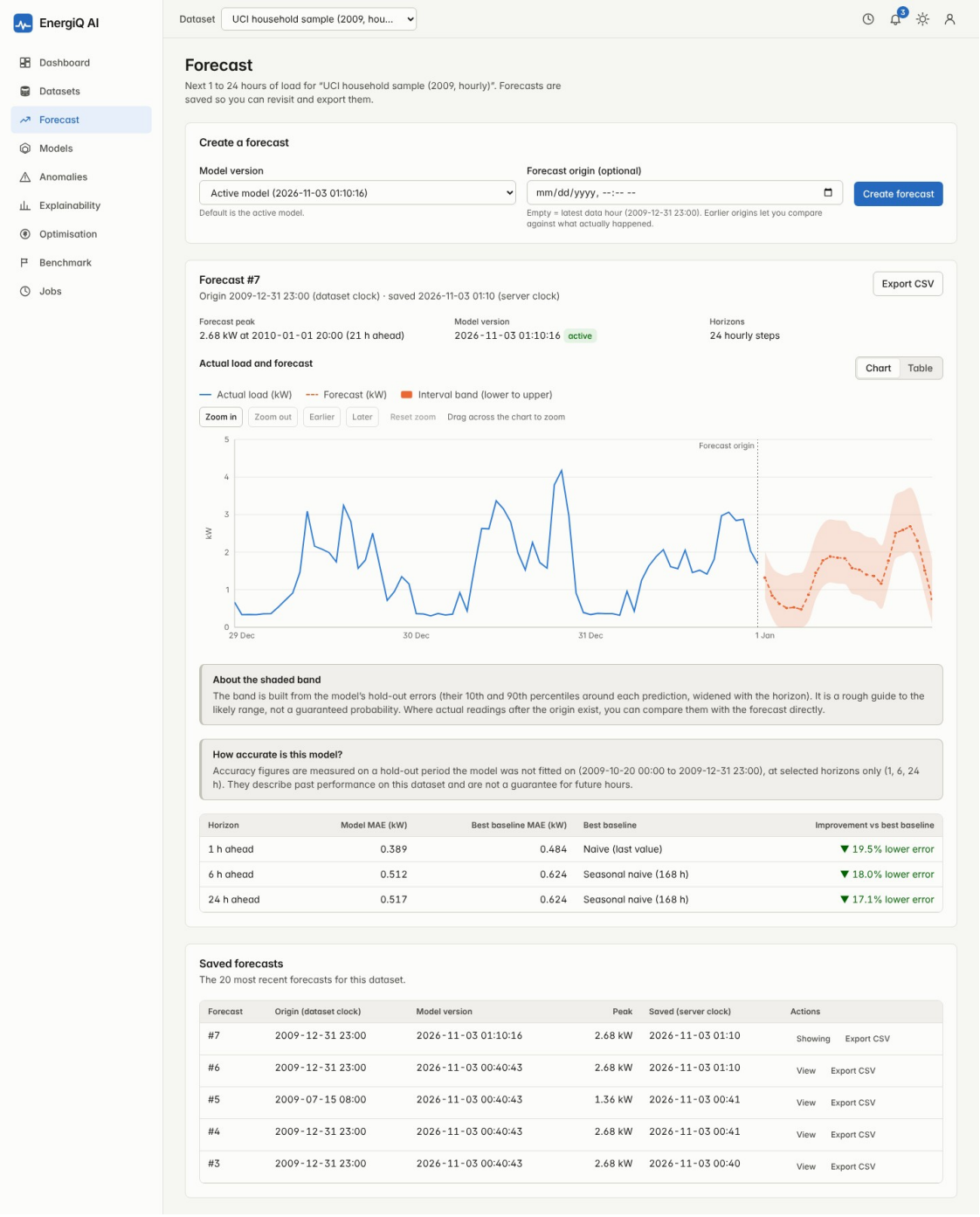
Dashboard



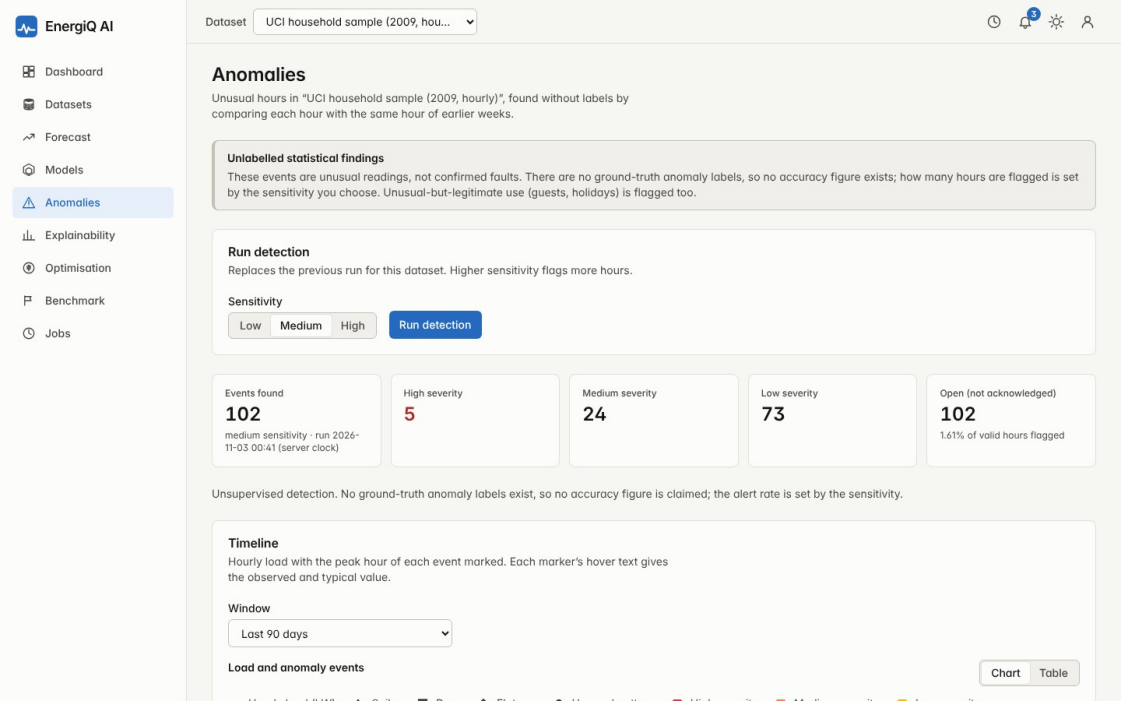
Dataset detail and quality report



Model versions and hold-out accuracy



Forecast with look-back origin



Anomaly detection

EnergiQ AI

Dataset: UCI household sample (2009, hou...)

- Dashboard
- Datasets
- Forecast
- Models
- Anomalies
- Explainability
- Optimisation
- Benchmark
- Jobs

Explainability

Why the model forecasts what it does. Explanations use exact tree SHAP values of the XGBoost forecaster.

Model version

2026-11-03 01:10:16 (active)

Global importance

Which inputs matter most across the hold-out period.

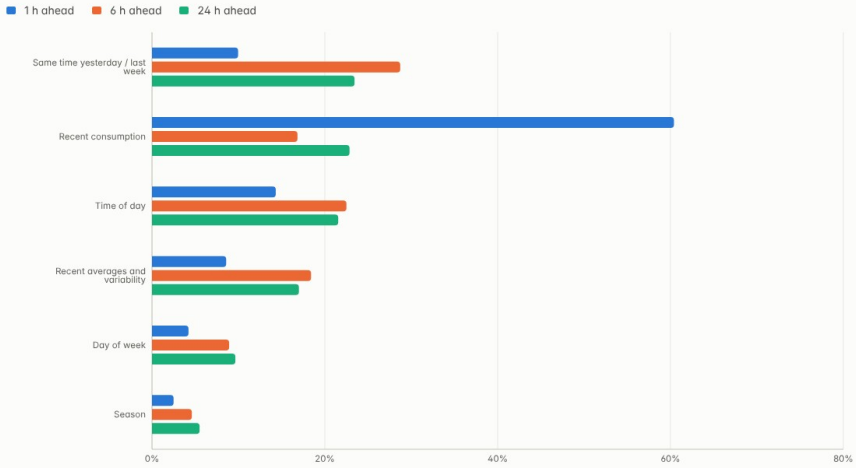
What this shows

mean |SHAP| on the hold-out period, per evaluated horizon. Each bar is the share of the model's total attribution that comes from a group of input features. It describes how the model behaves, not what causes consumption.

Share of importance by feature group

Percentage of total mean |SHAP|, per forecast horizon.

Chart | Table



Most important individual features

Horizon: 24 h ahead

Feature	Group	Mean |SHAP| (kW)	Share (%)
usage at the same time last week seasonal_168	Same time yesterday / last week	0.206	20.8
hour of day (number) hour	Time of day	0.192	19.5
variability over the last 7 days roll_std_168	Recent averages and variability	0.076	7.8
usage 11 h earlier lag_11	Recent consumption	0.052	5.3
usage 23 h earlier lag_23	Recent consumption	0.048	4.8
day of week (number) dow	Day of week	0.046	4.6
average of the last 7 days roll_mean_168	Recent averages and variability	0.044	4.5
usage in the latest hour lag_0	Recent consumption	0.040	4.1
usage 47 h earlier lag_47	Recent consumption	0.032	3.3
day of week (sine) dow_sin	Day of week	0.030	3.1

Local explanation

Break one specific forecast down into the inputs that pushed it up or down.

Horizon: 6 h ahead

Forecast origin (optional): mm/dd/yyyy, --:-- --

Empty = latest data hour (2009-12-31 23:00).

Explain this forecast

Forecast origin	Target hour	Forecast
2009-12-31 23:00	2010-01-01 05:00	0.478 kW

Average-case baseline	Additivity check
1.079 kW	contributions add up to the forecast within 1.2e-7 kW

The model forecasts 0.48 kW for 6 h ahead (average-case baseline 1.08 kW). Pushing it up: month (number) (+0.06 kW); average of the last 7 days (+0.05 kW). Pulling it down: hour of day (number) (-0.43 kW); usage at the same time last week (-0.20 kW).

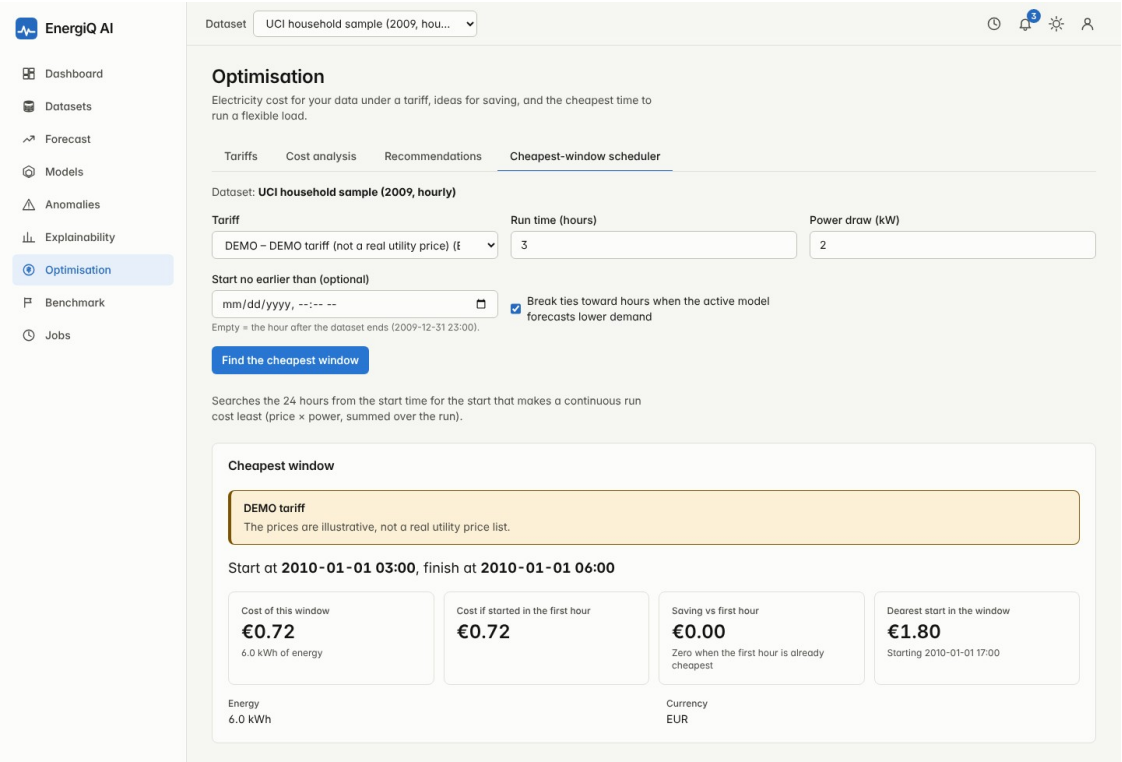
From baseline to forecast, by feature group

Each bar shows how much a group of inputs moves the forecast away from the average-case baseline.

Chart | Table



Explainability



Optimisation: cheapest-window scheduler

EnergiQ AI

Dataset: UCI household sample (2009, hou...)

- Dashboard
- Datasets
- Forecast
- Models
- Anomalies
- Explainability
- Optimisation
- Benchmark
- Jobs

Benchmark

How the forecasting methods compare on the public UCI household dataset. These are offline research results, not results for your own data.

Where these numbers come from

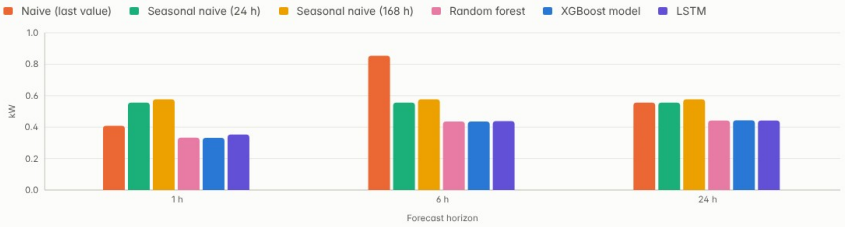
UCI Individual household electric power consumption (Hebrail & Berard, 2006), CC BY 4.0. Forecast errors are measured on the benchmark's hold-out period (2010-01-01 00:00 to 2010-11-26 23:00), which no model was fitted or tuned on. The figures below are shown as stored by the server.

Forecast comparison

Test-period MAE by method and horizon

Mean absolute error in kW on the benchmark test period. Lower is better.

Chart | Table



All metrics at one horizon — Horizon: 24 h ahead

Method	MAE (kW)	RMSE (kW)	sMAPE (%)	R^2
Naive (last value) best baseline in hindsight	0.556	0.821	49.8	-0.112
Seasonal naive (24 h)	0.556	0.821	49.8	-0.112
Seasonal naive (168 h) best baseline (cross-validation)	0.577	0.832	52.8	-0.143
Random forest	0.442	0.603	44.1	0.400
XGBoost model	0.443	0.606	44.2	0.394
LSTM	0.441	0.626	43.8	0.354

7,233 hours scored, 687 excluded (missing data). Mean actual load 1.055 kW.

Difference from the best cross-validated baseline (Seasonal naive (168 h)), 24 h ahead

Bootstrap over 312 test days. A negative MAE difference means the method has lower error than the baseline; the interval is 95%.

Method	MAE difference (kW)	95% interval (kW)	Interval excludes zero
Naive (last value)	−0.021	−0.047 to +0.005	no
Seasonal naive (24 h)	−0.021	−0.047 to +0.005	no
Random forest	−0.135	−0.153 to −0.119	yes
XGBoost model	−0.134	−0.152 to −0.116	yes
LSTM	−0.135	−0.156 to −0.116	yes

Head-to-head, 24 h ahead

Comparison (first minus second)	MAE difference (kW)	95% interval (kW)	Interval excludes zero
XGBoost model − Random forest	+0.002	−0.002 to +0.005	no
LSTM − XGBoost model	−0.002	−0.010 to +0.006	no

LSTM stability across random seeds (MAE, kW)

Seed	1 h	6 h	24 h	Best epoch
42 headline	0.353	0.438	0.441	11
43	0.362	0.451	0.451	6
44	0.351	0.441	0.447	8

Evaluation protocol

Target: hourly mean Global_active_power (kW), hours with >= 45 valid minutes

Test period: 2010-01-01 00:00 to 2010-11-26 23:00

Random seed: 42

Data fingerprint: 8b41e9a8111548ed

Fold	Validation start	Validation end
fold1	2008-01-01 00:00	2008-03-31 23:00
fold2	2008-04-01 00:00	2008-06-30 23:00
fold3	2008-07-01 00:00	2008-09-30 23:00
fold4	2008-10-01 00:00	2008-12-31 23:00
fold5	2009-01-01 00:00	2009-03-31 23:00
fold6	2009-04-01 00:00	2009-06-30 23:00
fold7	2009-07-01 00:00	2009-09-30 23:00
fold8	2009-10-01 00:00	2009-12-31 23:00

Anomaly detection benchmark

No ground-truth labels

The UCI data has no labelled anomalies, so there is no accuracy figure. The alert rate is a design setting of the detector.

Hours in series: 34,589

Valid hours: 34,151

Hours flagged: 695 (2.04% of valid hours)

References

Each entry was checked against a public source (publisher, proceedings, arXiv or repository metadata) on 3 November 2026. Publisher sites for several DOIs were unreachable during checking, so **click each DOI once before submission**. Two details need a manual look: the volume number of Cerqueira et al. (2020) was confirmed only indirectly, and sources disagree on the page range of Paszke et al. (2019) (8024–8035 in the PyTorch citation file, 8026–8037 in some indexes).

1. Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324
2. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). ACM. https://doi.org/10.1145/2939672.2939785
3. Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735
4. Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. In *Proceedings of the 8th IEEE International Conference on Data Mining* (pp. 413–422). IEEE. https://doi.org/10.1109/ICDM.2008.17
5. Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems, 30*. https://papers.nips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html
6. Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., & Lee, S.-I. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence, 2*(1), 56–67. https://doi.org/10.1038/s42256-019-0138-9
7. Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting, 22*(4), 679–688. https://doi.org/10.1016/j.ijforecast.2006.03.001
8. Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). OTexts. https://otexts.com/fpp3/
9. Bergmeir, C., & Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Information Sciences, 191*, 192–213. https://doi.org/10.1016/j.ins.2011.12.028
10. Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *Advances in Neural Information Processing Systems, 35* (Datasets and Benchmarks Track), 507–520.
11. Cerqueira, V., Torgo, L., & Mozetič, I. (2020). Evaluating time series forecasting models: An empirical study on performance estimation methods. *Machine Learning, 109*(11), 1997–2028. https://doi.org/10.1007/s10994-020-05910-7
12. Kong, W., Dong, Z. Y., Jia, Y., Hill, D. J., Xu, Y., & Zhang, Y. (2019). Short-term residential load forecasting based on LSTM recurrent neural network. *IEEE Transactions on Smart Grid, 10*(1), 841–851. https://doi.org/10.1109/TSG.2017.2753802
13. Hong, T., & Fan, S. (2016). Probabilistic electric load forecasting: A tutorial review. *International Journal of Forecasting, 32*(3), 914–938. https://doi.org/10.1016/j.ijforecast.2015.11.011

14. Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys, 41*(3), Article 15. https://doi.org/10.1145/1541880.1541882
15. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html
16. Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems, 32*. https://papers.nips.cc/paper/9015-pytorch
17. Hebrail, G., & Berard, A. (2006). *Individual household electric power consumption* [Data set]. UCI Machine Learning Repository. https://doi.org/10.24432/C58K54